

Министерство образования и науки Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра комплексной информационной безопасности
электронно-вычислительных систем (КИБЭВС)

Н.А. Новгородова

**ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ
MICROSOFT SQL SERVER**

**Учебно-методические указания к выполнению лабораторных работ
по дисциплинам
«Безопасность систем баз данных»,
«Базы данных и экспертные системы»,
«Базы данных»**

Томск 2015

ЛАБОРАТОРНАЯ РАБОТА №1

ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ MICROSOFT SQL SERVER

Цель работы: Реализация проекта учебной базы данных под управлением СУБД **Microsoft SQL Server**. Создание реляционных таблиц и установление межтабличных связей.

1 Краткие теоретические сведения

База данных (БД) — совокупность определенным образом организованных данных в рамках некоторой предметной области.

Например: база данных туристического агентства, библиотеки, отдела кадров, магазина и др.

Данные при этом могут храниться на разных носителях. Современным средством хранения и обработки баз данных является компьютер.

База данных - это объективная форма представления и организации совокупности данных, систематизированных таким образом, чтобы эти данные могли быть найдены и обработаны с помощью ЭВМ.

База данных представляет собой совокупность специальным образом организованных данных, хранимых в памяти вычислительной системы и отображающих состояние объектов и их взаимосвязей в рассматриваемой предметной области.

Система управления базой данных (СУБД) - специализированная программа (комплекс программ), предназначенная для организации и ведения БД.

СУБД предназначена для:

- создания БД;
- обеспечения эффективного доступа к данным в БД;
- управления данными в БД;
- поддержания БД в актуальном состоянии для предметной области;
- и др.

Microsoft SQL Server – система управления базами данных (СУБД).

Microsoft SQL Server – система управления реляционными базами данных (РСУБД).

Реляционная модель основана на математическом понятии **отношения** (англ. relation), наглядным представлением которого является **таблица**.

Реляционная база данных (РБД) представляет собой множество взаимосвязанных реляционных таблиц. Таблицы организованы в виде строк и столбцов. Каждая строка таблицы имеет одинаковую структуру и состоит из полей. Строкам таблицы соответствуют кортежи, а столбцам – атрибуты отношения.

Отношение/Реляционная таблица Сотрудники

Атрибут/Столбец/Колонка/Поле

Код сотрудника	Фамилия	Имя	Отчество	Должность
1	Петров	Иван	Иванович	Менеджер
2	Петрова	Ирина	Михайловна	Бухгалтер
3	Петров	Иван	Иванович	Менеджер

Кортеж/Строка/Запись

Рисунок 1.1 – Пример реляционной таблицы БД

Каждая реляционная таблица БД хранит данные об определенном объекте (сущности) предметной области.

Объекты (сущности) – это вовлеченные в деятельность люди, предметы, события, документы, места и т.п., информация о которых должна накапливаться в разрабатываемой БД и которые представляют интерес для пользователей.

На поля в таблицах БД могут быть наложены ограничения. Например:

- уникальность значений;
- значения по умолчанию;
- обязательное указание;
- диапазон значений;
- список значений;
- условия на значения.

Для однозначного определения каждой записи таблица должна иметь **первичный ключ**.

Первичный ключ – это атрибут или набор атрибутов, значения в которых позволяют однозначно определять запись в таблице.

Требования к первичному ключу:

- должен однозначно определять строку таблицы;
- должен быть уникален;
- не должен содержать пустых значений (должен быть обязательно указан, NOT NULL);
- не должен быть избыточным.

Выбор первичного ключа лежит на разработчике БД.

На рисунке 1.1 в качестве первичного ключа может быть выбрано поле **Код сотрудника** при условии, что значения в данном поле будут уникальны, не будут пустыми. Поля по отдельности Фамилия, Имя, Отчество, Должность не могут быть выбраны в качестве первичного ключа, так как могут содержать повторяющиеся значения. Совокупность полей Фамилия, Имя, Отчество, Должность так же не могут быть выбраны в качестве первичного ключа, так как могут быть полные однофамильцы на одной и той же должности.

Связи между реляционными таблицами БД соответствуют связям между объектами (сущностями) предметной области.

Объекты (сущности) предметной области **состоят в связи**, если хотя бы одному экземпляру одного из них можно поставить в соответствии (по определенному правилу) один или более экземпляров другого.

Для реляционных таблиц БД: реляционные таблицы БД **состоят в связи**, если хотя бы одной записи одной из них можно поставить в соответствии (по определенному правилу) одну или более записей в другой таблице.

Внешний ключ (анг.foreign key (FK)) – поле или набор полей, предназначенное для организации связи между данными в таблицах БД.

Внешний ключ - это ссылка полей одной таблицы на поля другой таблицы.

При установке связей между таблицами необходимо помнить про **ссылочную целостность**.

Для ссылочной целостности характерно:

- первичный ключ (PK) таблицы должен содержать уникальные непустые значения для данной таблицы;
- внешний ключ (FK) потомка должен содержать только те значения, которые уже имеются среди значений PRIMARY KEY родителя (или колонок UNIQUE родителя);
- нельзя удалить экземпляр родителя, имеющий хотя бы один связанный с ним экземпляр потомка. Чтобы удалить экземпляр родителя, имеющего связанные потомки, необходимо сначала удалить связанные записи в потомке, а потом удалить экземпляр родителя с ними связанного. ИЛИ Можно установить механизм **каскадного удаления**, при котором удаляется экземпляр родителя вместе со связанными экземплярами потомка;
- нельзя изменить значение в PK (UNIQUE) родителя, имеющем хотя бы один связанный с ним экземпляр потомка. Можно установить механизм **каскадного обновления**, при котором изменяется PK (UNIQUE) родителя и значения FK связанных экземпляров потомка.

Ограничение FOREIGN KEY может содержать пустые значения (значения NULL). Кроме того, тип данных и размер колонки или колонок внешнего ключа должны совпадать со ссылочной колонкой или колонками.

Внешний ключ в таблице может ссылаться и на саму эту таблицу. В таких случаях говорят о **рекурсивном внешнем ключе**. Рекурсивные внешние ключи используются для хранения древовидной структуры данных в реляционной таблице.

Связь характеризуется мощностью связи.

Мощностью связи называется максимальное количество записей одной таблицы БД, связанных с одной записью другой таблицы БД.

Наиболее распространены мощности связи:

- один к одному 1:1;
- один ко многим 1:M;
- многие ко многим M:M.

Связь 1:1: одной записи в одной таблице соответствует одна запись в другой таблице.

Например: **Сотрудники 1 : 1 Дополнительно**

Сотрудники				
Код сотрудника	Фамилия	Имя	Отчество	Должность
1	Петров	Иван	Иванович	Менеджер
2	Сидорова	Ирина	Петровна	Бухгалтер
3	Петров	Иван	Иванович	Менеджер

Дополнительно		
Код сотрудника (FK)	Адрес	Паспортные данные
1	г. Томск	123456 и далее
2	г. Сургут	654321 и далее
3	г. Томск	789123 и далее

Связь 1:M: наиболее распространена. Одной записи в одной таблице (родительской) соответствует несколько записей в другой таблице (дочерней, потомок). Запись в дочерней таблице связана с одной записью в родительской таблице.

Например: **Сотрудники 1:M Продажи**

Сотрудники				
Код сотрудника	Фамилия	Имя	Отчество	Должность
1	Петров	Иван	Иванович	Менеджер
2	Сидорова	Ирина	Петровна	Бухгалтер
3	Петров	Иван	Иванович	Менеджер

Продажи		
Номер продажи	Дата	Код сотрудника (FK)
1П	12.02.2008	1
2П	12.02.2008	1
3П	12.02.2008	2
4П	20.02.2008	3
5П	21.02.2008	1

Связь M:M - одной записи первой таблицы может соответствовать несколько записей во второй таблице. И одной записи второй таблицы может соответствовать несколько записей в первой таблице.

Например: **Продажи M:M Блюда**

Продажи		
Номер продажи	Дата	Код сотрудника (FK)
1П	12.02.2008	1
2П	12.02.2008	1
3П	12.02.2008	2
4П	20.02.2008	3
5П	21.02.2008	1

Блюда				
Код блюда	Название	Тип	Цена меню	Выход
1Б	Оливье	Салаты	100 руб.	250 гр.
2Б	Борщ	Супы	90 руб.	300 гр.
3Б	Цезарь	Салаты	120 руб.	270 гр.

Для моделирования связи М:М в БД вводится дополнительная связующая таблица, которая разбивает связь М:М на связи 1:М и М:1:

Продажи		
Номер продажи	Дата	Код сотрудника (FK)
1П	12.02.2008	1
2П	12.02.2008	1
3П	12.02.2008	2
4П	20.02.2008	3
5П	21.02.2008	1

Блюда				
Код блюда	Название	Тип	Цена меню	Выход
1Б	Оливье	Салаты	100 руб.	250 гр.
2Б	Борщ	Супы	90 руб.	300 гр.
3Б	Цезарь	Салаты	120 руб.	270 гр.

Продано				
Номер продажи (FK)	Код блюда (FK)	Количество	Цена	Скидка
1П	1Б	2	100 руб.	0%
1П	3Б	1	120 руб.	0%
2П	2Б	10	90 руб.	10%
3П	1Б	3	100 руб.	0%
3П	2Б	1	90 руб.	0%
3П	3Б	1	120 руб.	5%

Вторичные ключи устанавливаются по полям, которые часто используются при поиске или сортировке данных. Построенные по вторичным ключам индексы помогут системе значительно быстрее найти нужные значения, хранящиеся в соответствующих полях. В отличие от первичных ключей, поля для вторичных ключей могут содержать не уникальные значения.

Например, можно проиндексировать в таблице Блюда поля Тип, Название и др.

2 Особенности работы с Microsoft SQL Server

СУБД Microsoft SQL Server основана на стандарте SQL с некоторыми добавленными компанией Microsoft расширениями. Microsoft SQL Server распознает язык **Transact-SQL**, или, сокращенно, T-SQL.

Существуют различные инструменты, которые позволяют управлять Microsoft SQL Server, преобразуя щелчки мышью на определенных инструментах в соответствующие команды T-SQL, отправляемые ядру базы данных. Например, в **Microsoft Visual Studio, SQL Server Management Studio** и др., которые позволяют разработчикам и администраторам баз данных создавать проекты баз данных и управлять СУБД SQL Server с помощью графического интерфейса пользователя либо с помощью инструкций T-SQL.

В лабораторной работе описываются возможности SQL Server Management Studio для реализации проекта учебной базы данных под управлением СУБД Microsoft SQL Server.

Среда **SQL Server Management Studio** — это интегрированная среда для доступа, настройки, администрирования, разработки компонентов/объектов Microsoft SQL Server и управления ими.

В среде SQL Server Management Studio графические средства сочетаются с набором функциональных редакторов скриптов для доступа разработчиков и администраторов с разным уровнем знаний к SQL Server.

Главным инструментом SQL Server Management Studio является **Object Explorer (Обозреватель объектов)**, который позволяет пользователю просматривать, извлекать объекты сервера, а также ими управлять.

Также есть **SQL Server Management Studio Express** для Express версии сервера, которая является бесплатной. Однако в ней нет поддержки ряда компонентов (Analysis Services, Integration Services и др.).

Среда **Microsoft SQL Server Management Studio Express** — это бесплатная интегрированная среда для доступа, настройки, управления, администрирования и разработки всех компонентов SQL Server, а также объединение простых в использовании графических средств и редакторов с богатыми сценарными возможностями для доступа к SQL Server разработчиков и администраторов с любым уровнем подготовки.

Для доступа к Microsoft SQL Server необходимо наличие у пользователя соответствующей учетной записи для аутентификации.

Microsoft SQL Server предполагает использование двух различных режимов аутентификации:

- **регистрационные записи SQL Server** - учетные записи создаются и управляются в самой СУБД. Эти записи могут быть использованы только для доступа к серверу баз данных;

- **учетные записи Windows** - учетные записи Windows создаются для сетевого доступа и могут быть использованы также для доступа к серверу баз данных.

Использование для доступа к Microsoft SQL Server обоих методов называют **смешанным режимом**.

При использовании аутентификации Windows Microsoft SQL Server запрашивает регистрационные данные пользователя у операционной системы. Для регистрации пользователей в Windows используется подсистема Учетные записи пользователей панели управления.

Аутентификация Windows открывает пользователям доступ к базе данных, проверяя их регистрационные данные относительно учетных записей операционной системы. Это позволяет централизовать включение, отключение и применение особых настроек системы безопасности, связанных с учетными записями пользователей. Если какой-либо работник увольняется из организации, то отключение его учетной записи Windows автоматически закрывает ему доступ и к базе данных.

В Microsoft SQL Server можно создать собственные регистрационные записи. Создание отдельной учетной записи для служб Microsoft SQL Server обеспечит большую защиту и надежность, так как при этом можно снабдить ее минимальным набором необходимых разрешений, а недоступность пароля для обычных пользователей уменьшит возможность воспользоваться учетной записью службы для удовлетворения собственных потребностей.

2.1 База данных Microsoft SQL Server

База данных Microsoft SQL Server представляет собой одну логическую единицу, даже несмотря на то, что физически может размещаться в нескольких файлах.

Для создания БД можно использовать команду DDL **CREATE DATABASE** с соответствующими параметрами или воспользоваться соответствующим интерфейсом.

База данных состоит из двух файлов (точнее, из двух наборов файлов):

- файл данных с заданным именем и расширением **.mdf**;
- файл журнала транзакций с расширением **.ldf**.

Файл данных содержит системные и пользовательские таблицы, индексы, представления, хранимые процедуры, пользовательские функции, триггеры и разрешения системы безопасности.

Журнал транзакций с последовательной записью является «центром» SQL Server. Все обновления данных вначале записываются в журнал транзакций и проверяются, что гарантирует запись обновлений в двух

местах. Журнал транзакций содержит не только пользовательские записи, но и индексные записи, разбиения страниц, реорганизации таблиц и многое другое. Файл транзакций содержит не только текущую информацию, но и все обновления файла данных, он, как правило, все время разрастается.

Файлы данных и журнала транзакций по умолчанию размещаются в каталоге, принятом в SQL Server по умолчанию. Место размещения файлов данных и журнала транзакций определяется в окне параметров сервера.

Подобно всем реляционным СУБД, Microsoft SQL Server является таблично ориентированной. После создания базы данных следующим действием является наполнение ее таблицами.

Можно создавать таблицы, используя DDL **CREATE TABLE** или работать в графической среде, например в Visual Studio, SQL Management Studio.

При этом можно создавать таблицы в так называемом конструкторе таблиц (проект таблицы).

2.2 Имена таблиц и столбцов

Microsoft SQL Server допускает использование в именах таблиц и столбцов до 128 символов Unicode, включая пробелы, а также символы верхнего и нижнего регистров.

Если в названии используются пробелы, то в дальнейшем придется заключать их в квадратные скобки. Пробел можно заменить, например, символом подчеркивания или комбинированные слова типа OrderDetail, набранные в верхнем и нижнем регистре. Имена необходимо задавать согласно их смыслу.

2.3 Конструктор таблиц

Конструктор таблиц - визуальный инструмент, который позволяет проектировать и визуализировать таблицы базы данных, к которой подключены. Можно использовать конструктор таблиц для создания, редактирования и удаления таблиц, столбцов, ключей, индексов, связей и ограничений.

Конструктор таблиц позволяет создавать новые таблицы, а также модифицировать структуру уже существующих. Интерфейс этого окна схож с интерфейсом аналогичного окна конструкторов прочих приложений.

Открыть конструктор таблиц можно путем создания новой таблицы или открытия существующей таблицы.

2.3.1 Создание новой таблицы

1. Щелкните правой кнопкой мыши на узле **Tables (Таблицы)** в обозревателе объектов.

2. Из выпадающего меню выберите пункт Создать таблицу.
3. После этого откроется конструктор таблицы, в котором можно создавать необходимую структуру.
4. Для таблицы введите имена столбцов, выберите типы данных, а также выберите допустимы ли пустые значения для каждого столбца. Кроме того можно установить необходимые ограничения.
5. Из меню Файл (**File**) выберите Сохранить имя таблицы (**Save table name**).
6. В диалоговом окне введите имя таблицы и нажмите кнопку **ОК**.

2.3.2 Отображение свойств таблицы в окне «Свойства»:

1. Щелкните правой кнопкой мыши таблицу в конструкторе таблиц.
2. Выберите Свойства (**Properties**) из контекстного меню.

Например, **свойства таблицы для Microsoft SQL Server**:

- **Name** (Имя) - отображает имя таблицы.
- **Database Name** (Название базы данных) - показывает имя базы данных выбранной таблицы.
- **Description** (Описание) - показывает описание выбранной таблицы. Чтобы увидеть полное описание или изменить его, нажмите **Description** и нажмите кнопку с многоточием (...) справа от свойства.
- **Server** (Сервер) - показывает имя сервера для источника данных.
- **Schema** (Схема) - показывает имя схемы, которой принадлежит эта таблица.
- **Is Replicated** (Реплицируется) - показывает, может ли таблица воспроизводиться в другом месте.
- и др.

В конструкторе таблицы каждый столбец таблицы выглядит как строка в сетке, а в столбцах сетки представляют **свойства столбца таблицы**.

2.3.3 Свойства поля /столбца (Column Properties)

Свойства можно настраивать в разделе Свойства столбца (**Column Properties**) в конструкторе таблиц. Чтобы изменить свойства столбцов для баз данных Microsoft SQL Server, выберите столбец в конструкторе таблиц.

Например, **свойства столбца**:

- **Name** (Имя) - показывает название столбца.
- **Database** (База данных) - показывает название базы данных для выбранного столбца.
- **Data Type** (Тип данных) - отображает тип данных выбранного столбца.

- **Is Identity** (идентификатор) – показывает, является ли выбранный столбец идентификатором таблицы.

- **Length** (Длина) - показывает количество символов, допустимое для символьных типов данных.

- **Precision** (Точность) - показывает максимальное число знаков, допустимых для числовых типов данных. Это свойство показывает, 0 для нечисловых типов данных.

- **Scale** (Масштаб) - показывает максимальное количество цифр, которые могут появляться справа от десятичной точки для числовых типов данных. Это свойство показывает, 0 для нечисловых типов данных.

- **Allow Nulls** (Разрешить NULL значения) - показывает, можете ли или нет данный столбец содержать NULL значения.

- **Default Value** (Значение по умолчанию) – значения по умолчанию.

- **Formula** (Формула) - показывает формулу, которую использует выбранный столбец.

- **Description** (Описание) - отображает описание столбца. Чтобы увидеть полное описание или изменить его, нажмите Описание и нажмите кнопку с многоточием (...) справа от свойства.

- **Language** (Язык) - указывает язык, используемый для индексирования столбца.

- **Identity Increment** (Прирост идентификатора) - показывает прирост, который будет добавлен для каждой последующей строки. Если оставить эту ячейку пустой, значение 1 будет назначено по умолчанию.

- **Identity Seed** (начальное значение идентификатора) - показывает значение, присвоенное в первой строке в таблице. Если оставить эту ячейку пустой, значение 1 будет назначено по умолчанию.

- **Is Indexable** (Индексируется) - показывает, имеются ли в выбранном столбце данные, которые могут быть проиндексированы.

- **Is Replicated** (Реплицируется) - показывает, может ли этот столбец воспроизведен в другом месте.

- **Size** (Размер) - показывает размер в байтах для типа данных столбца. Например, NCHAR тип данных может иметь длину от 10 (количество символов), но это будет иметь размер 20 на счет для наборов символов Юникода.

- и др.

2.3.4 Вставка столбцов в таблицу в конструкторе таблицы

1. Откройте необходимую таблицу в конструкторе таблиц.
2. Щелкните правой кнопкой мыши строку в таблице и выберите **Insert Column** (Вставить столбец) из контекстного меню. Пустая строка столбец вставляется.

или

поместите курсор в первую пустую ячейку в столбце **Column Name** (Имя столбца).

3. В столбце **Column Name** (Имя столбца) введите имя столбца в ячейке.
4. Нажмите клавишу **TAB** для перехода к ячейке Тип данных (**Data Type**) и выберите тип данных из выпадающего меню.
5. Продолжите определять какие-либо другие свойства столбца во вкладке **Properties** (Свойства) столбца.

2.3.5 Удаление столбцов из таблицы

1. Выберите столбец, который необходимо удалить.
2. Щелкните правой кнопкой мыши столбец и выберите команду **Delete Column** (Удалить столбец) из контекстного меню.
3. Если столбец участвует в связях, сообщение попросит вас подтвердить удаление выбранных столбцов и их связей.

Если столбец не участвует в проверочном ограничении связи, то столбец будет удален из базы данных.

Если в столбце делается проверочное ограничение связи, то сервер базы данных отклонит изменения, когда вы пытаетесь сохранить свою работу. Чтобы удалить столбец, который участвует в проверочном ограничении связи, необходимо сначала изменить или удалить проверочное ограничение.

2.4 Типы данных столбцов в Microsoft SQL Server

Тип данных столбца служит для двух целей:

- обеспечивает первый уровень целостности данных. Символьные данные не могут быть занесены в столбцы типов даты-времени и числового типа;
- определяет объем дискового пространства, выделяемого для столбца.

2.4.1 Символьные типы данных

SQL Server поддерживает несколько символьных типов данных:

- **char (n)** – символьные данные фиксированной длины, не в Юникоде, с длиной *n* байт. Значение *n* должно находиться в интервале от 1 до 8000. Размер хранения данных этого типа равен *n* байт. Синонимом по стандарту ISO для типа **char** является **character**.

- **varchar (n | max)** - символьные данные переменной длины, не в Юникоде. *n* может иметь значение от 1 до 8 000. **max** означает, что максимальный размер хранения равен $2^{31}-1$ байт. Размер хранения равен фактической длине данных плюс два байта. Введенные данные могут иметь длину 0 символов. Синонимами по стандарту ISO для типа **varchar** являются типы **char varying** или **character varying**.

- **nchar (n)** - символьные данные фиксированной длины в Юникоде. Аргумент *n* должен иметь значение от 1 до 4000. Размер хранилища вдвое

больше n байт. Синонимами по стандарту ISO для типа **nchar** являются типы **national char** и **national character**.

- **nvarchar** (n | **max**) - символьные данные в Юникоде переменной длины. Аргумент n может принимать значение от 1 до 4 000. Аргумент **max** указывает, что максимальный размер хранилища равен $2^{31}-1$ байт. Размер хранилища в байтах вдвое больше числа введенных символов + 2 байта. Введенные данные могут иметь длину в 0 символов. Синонимами по стандарту ISO для типа **nvarchar** являются типы **national char varying** и **national character varying**.

- **ntext** - представляет символьные данные в Юникоде переменной длины, включающие до $2^{30} - 1$ (1 073 741 823) символов. Объем занимаемого этим типом пространства (в байтах) в два раза превышает число символов. Синонимом по стандарту ISO для типа **ntext** является **national text**.

- **text** - представляет данные, отличные от данных Юникод, представленные с использованием кодовой страницы сервера. Максимальная длина данных — $2^{31} - 1$ (2 147 483 647) символов. Если в кодовой странице сервера используются двухбайтовые символы, объем занимаемого типом пространства все равно не превышает 2 147 483 647 байт. Он может быть менее 2 147 483 647 байт — в зависимости от строки символов.

2.4.2 Числовые типы данных

SQL Server поддерживает несколько числовых типов данных.

- **bit** - целочисленный тип данных, который может принимать значения 1, 0 или NULL.

Типы точных числовых данных, использующие **целые значения**:

Тип данных	Диапазон	Хранилище
bigint	от -2^{63} (-9 223 372 036 854 775 808) до $2^{63}-1$ (9 223 372 036 854 775 807)	8 байт
int	от -2^{31} (-2 147 483 648) до $2^{31}-1$ (2 147 483 647)	4 байта
smallint	от -2^{15} (-32 768) до $2^{15}-1$ (32 767)	2 байта
tinyint	от 0 до 255	1 байт

- **decimal**[(p [, s])] и **numeric**[(p [, s])] - числа с фиксированной точностью и масштабом. При использовании максимальной точности числа могут принимать значения в диапазоне от $-10^{38}+1$ до $10^{38}-1$. В стандарте ISO синонимом типа **decimal** являются типы **dec** и **dec**(p , s). Тип **numeric** функционально равнозначен типу **decimal**.

p (точность)

Максимальное количество десятичных разрядов числа (как слева, так и справа от десятичной запятой). Точность должна принимать значение от 1 до 38. По умолчанию для точности принимается значение 18.

s (масштаб)

Максимальное количество десятичных разрядов числа справа от десятичной запятой. Масштаб может принимать значение от 0 до p . Масштаб может быть указан только совместно с точностью. По умолчанию масштаб принимает значение 0; поэтому $0 \leq s \leq p$. Максимальный размер хранилища зависит от точности.

Точность Байты хранилища

1 - 9	5
10-19	9
20-28	13
29-38	17

Типы данных, представляющие денежные (валютные) значения:

Тип данных	Диапазон	Хранение
Money	От -922 337 203 685 477,5808 до 922 337 203 685 477,5807	8 байт
smallmoney	От -214 748,3648 до 214 748,3647	4 байта

Типы приблизительных числовых данных, используемые для числовых данных с плавающей запятой. Данные с плавающей запятой являются приблизительными, поэтому не все значения из диапазона могут быть отображены точно.

Тип данных	Диапазон	Хранение
Float	- 1,79E+308 — -2,23E-308, 0 и 2,23E-308 — 1,79E+308	Зависит от значения n
Real	- 3,40E + 38 — -1,18E - 38, 0 и 1,18E - 38 — 3,40E + 38	4 байта

float [(n)]

Где n — это количество битов, используемых для хранения мантиссы числа в формате **float** при экспоненциальном представлении.

Определяет точность данных и размер для хранения. Значение параметра n должно лежать в пределах от 1 до 53. Значением по умолчанию для параметра n является 53.

n Точность Объем памяти

1–24	7 знаков	4 байта
------	----------	---------

2.4.3 Типы данных даты и времени

Тип данных	Format	Диапазон	Точность	Объем памяти (в байтах)	Определя емая пользоват елем точность в долях секунды	Смещ ение часово го пояса
Time	чч:мм:сс[.нннннн]	От 00:00:00.00000 00 до 23:59:59.99999 99	100 наносекун д	От 3 до 5	Да	Нет
date	ГГГГ-ММ-ДД	От 0001-01-01 до 9999-12-31	1 день	3	Нет	Нет
smalldatetime	ГГГГ-ММ-ДД чч:мм:сс	От 01.01.1900 до 06.06.2079	1 минута	4	Нет	Нет
datetime	ГГГГ-ММ-ДД чч:мм:сс[.ннн]	От 01.01.1753 до 9999-12-31	0,00333 секунда	8	Нет	Нет
datetime2	ГГГГ-ММ-ДД чч:мм:сс[.нннннн]	От 0001-01-01 00:00:00.00000 00 до 9999-12- 31 23:59:59.99999 99	100 наносекун д	От 6 до 8	Да	Нет
datetimeoffset	ГГГГ-ММ-ДД чч:мм:сс[.нннннн] [+ -]чч:мм	От 0001-01-01 00:00:00.00000 00 до 9999-12- 31 23:59:59.99999 99 (время в формате UTC)	100 наносекун д	От 8 до 10	Да	Да

Функции, получающие компоненты даты и времени:

Функция	Синтаксис	Возвращаемое значение	Тип возвращаемых данных
DATENAME	DATENAME (<i>datepart, date</i>)	Возвращает строку символов, представляющую указанную часть <i>datepart</i> заданной даты.	nvarchar
DATEPART	DATEPART (<i>datepart, date</i>)	Возвращает целое число, представляющее указанный компонент <i>datepart</i> указанной даты <i>date</i> .	int
DAY	DAY (<i>date</i>)	Возвращает целое число,	int

		представляющее день указанной даты <i>date</i> .	
MONTH	MONTH (<i>date</i>)	Возвращает целое число, представляющее месяц указанной даты <i>date</i> .	int
YEAR	YEAR (<i>date</i>)	Возвращает целое число, представляющее год указанной даты <i>date</i> .	int

Функции системной даты и времени меньшей точности:

Функция	Синтаксис	Возвращаемое значение	Тип возвращаемых данных
CURRENT_TIMESTAMP	CURRENT_TIMESTAMP	Возвращает значение типа datetime2(7) , которое содержит дату и время компьютера, на котором запущен экземпляр SQL Server. Смещение часового пояса не включается.	datetime
GETDATE	GETDATE ()	Возвращает значение типа datetime2(7) , которое содержит дату и время компьютера, на котором запущен экземпляр SQL Server. Смещение часового пояса не включается.	datetime
GETUTCDATE	GETUTCDATE ()	Возвращает значение типа datetime2(7) , которое содержит дату и время компьютера, на котором запущен	datetime

экземпляр SQL

Server.

Возвращаемые дата и

время отображаются

в формате UTC.

2.4.4 Прочие типы данных

- **image** - этот тип представляет двоичные данные переменной длины, включающие от 0 до $2^{31} - 1$ (2 147 483 647) байт.

Типы двоичных данных фиксированной или переменной длины:

- **binary** [(*n*)] - двоичные данные фиксированной длины размером в *n* байт, где *n* — значение от 1 до 8000. Размер хранения составляет *n* байт.

- **varbinary** [(*n* | **max**)] - двоичные данные переменной длины. *n* могут иметь значение от 1 до 8000; **max** означает максимальную длину хранения, которая составляет $2^{31}-1$ байт. Размер хранения — это фактическая длина введенных данных плюс 2 байта. Введенные данные могут иметь размер 0 символов. В ANSI SQL синонимом для **varbinary** является **binary varying**.

Тип **binary** применяется в тех случаях, когда размер данных в столбце поддается предварительной оценке.

Если же размер данных в столбце заранее определить трудно, пользуйтесь типом **varbinary**.

Если элементы данных в столбце превышают в длину 8000 байт, пользуйтесь типом данных **varbinary(max)**.

Тип данных, в котором хранятся XML-данные. Можно хранить экземпляры **xml** в столбце либо в переменной типа **xml**.

Примечание:

Типы данных **ntext**, **text** и **image** в будущей версии Microsoft SQL Server будут удалены. Следует избегать их использования при разработке новых приложений и запланировать изменение приложений, в которых эти типы в данное время используются. Вместо этих типов данных следует использовать типы **nvarchar(max)**, **varchar(max)** и **varbinary(max)**.

2.5 Работа с индексами

Индекс представляет собой механизм, обеспечивающий быстрый доступ к строкам таблицы или для соблюдения определенных ограничений.

2.6 Ограничение уникальности

2.6.1 Создание ограничения уникальности

1. Выберите таблицу, к которой вы хотите добавить ограничение уникальности.
2. Откройте таблицу в конструкторе таблиц.
3. В меню конструктора таблицы выберите **Indexes/Keys**.
4. В диалоге **Indexes/Keys** нажмите кнопку **Add**.
5. В сетке выберите тип (**Type**) и выберите уникальный ключ (**Unique Key**) из раскрывающегося списка справа от свойства.
6. Ограничение уникальности, созданные в базе данных, сохраняются при сохранении таблицы.

2.6.2 Удаление ограничения уникальности

1. В выбранной таблице в режиме конструктора выберите **Indexes/Keys**.
2. В диалоге **Indexes/Keys** выберите уникальный индекс в **Selected Primary/Unique Key and Index list**.
3. Нажмите кнопку **Delete**.
4. Ограничение удаляется из базы данных при сохранении таблицы.

2.7 Ограничение первичного ключа

Ограничение первичного ключа гарантируют уникальность первичного ключа и отсутствие в нем пустых значений.

Таблица может иметь только один первичный ключ. Первичный ключ может быть составным. Если первичный ключ состоит более чем из одного столбца, повторяющиеся значения допускаются в один столбец, но каждая комбинация значений всех столбцов в первичном ключе должны быть уникальными.

2.7.1 Создание первичного ключа

1. В конструкторе таблицы нажмите кнопку выделения строки для столбца, который необходимо определить в качестве первичного ключа. Если хотите выбрать несколько столбцов, нажмите клавишу CTRL для других столбцов.
2. Щелкните правой кнопкой мыши выделения строки для столбца и выберите **Set Primary Key** (Задать первичный ключ). Первичный ключ индекса, под названием "ПК_", а затем имя таблицы, создается автоматически, вы можете найти его в диалоговом окне Индексы /Ключи (**Indexes/Keys**).

2.7.2 Удаление ограничения первичного ключа

1. В режиме конструктора таблицы щелкните правой кнопкой мыши на строке с первичным ключом и выберите **Remove Primary Key** (Удалить первичный ключ).

2.8 Свойства для столбца идентификатора автоинкремента (счетчика)

1. Откройте в режиме конструктора необходимую таблицу.
2. Уберите флаг **Allow nulls** для столбца, который нужно изменить.
3. В свойствах столбца (**Column Properties**) раскройте свойство **Identity Specification**.
4. Щелкните ячейку сетки **Is Identity** и выберете **Yes** (Да) в раскрывающемся списке.
5. Введите значение в ячейке **Identity Seed**, которое будет назначено на первую строку в таблице. 1 будет назначен по умолчанию.
6. Введите значение в ячейке **Identity Increment** значение приращения автоинкремента. Это значение приращения, которые будут добавлены для каждой последующей записи. 1 будет назначен по умолчанию.

IDENTITY [(seed , increment)]

Аргументы:

seed Значение, присваиваемое самой первой строке, загружаемой в таблицу.

increment Значение приращения, которое прибавляется к значению идентификатора предыдущей загруженной строки.

Необходимо указывать либо оба аргумента (и seed, и increment), либо не указывать ни одного из них. Если ничего не указано, применяется значение по умолчанию (1,1).

2.9 Значение по умолчанию

Считаются свойством столбца, так как никаких проверок ввода данных пользователем не проводится, также не форсируется поддержание целостности данных. В то же время значения, заданные по умолчанию, являются ценным параметром столбца.

Значение по умолчанию может быть одним из следующих:

- допустимое статическое числовое или символьное значение;
- системная функция, такая как GetDate() и др.
- функция, определенная пользователем;
- пустое значение.

Тип значения по умолчанию должен быть совместим с типом данных столбца.

Указание значения по умолчанию для столбца:

1. Выберите столбец, для которого вы хотите задать значение по умолчанию.
2. В колонке на вкладке **Properties** (Свойства) введите новое значение по умолчанию **Default Value or Binding**.

Если поле строкового типа, то значение по умолчанию для строк текста, заключите в одинарные кавычки ('). Чтобы ввести числовые значения по умолчанию, введите число. Для ввода объекта /функции можете ввести имя объекта / функции без единой кавычки.

2.10 Допустимость NULL значений

NULL значение можно расценивать как неизвестное. Как правило, NULL значения появляются при неполном вводе пользователем данных в строку. Допустимы ли в столбце NULL значения, определяется с помощью атрибута столбца NULL или NOT NULL

Если NULL значения не допускаются, пользователю, который вводит данные в таблице, необходимо ввести значение в поле, так как запись не может быть сохранена в базе данных.

Изменение параметров по NULL столбцу:

1. Выберите столбец и на вкладке **Properties** (Свойства) колонке установите или уберите флаг **Allow nulls** (допускать NULL значения).

2.11 Ограничение проверки

Ограничения проверки (**check constraint**) являются проверкой целостности данных на уровне строк.

Могут быть диапазон, список, условия на значения и др. Как правило, это небольшая формула, которая возвращает булево значение true или false. В ограничения проверки могут быть включены такие операторы, как =, >, <, >=, <=, BETWEEN, LIKE, IN и др. Можно создать сложное выражение, используя логические операторы, чтобы проверить данные по нескольким условиям. Ограничения проверки могут содержать пользовательские функции, которые могут выполнять последовательность инструкций. Ограничения проверки полезны для поддержания общих правил допустимости данных, а также простых правил бизнес-логики.

Примеры операторов:

Оператор	Описание	Пример
=	Равно	Цена =1000 записи с ценой равной

		1000 Тип = 'Фрукты' записи с типом фрукты															
<>	Не равно	Количество <>500 записи с количеством не равным 500															
>	Больше	Цена >1000 записи с ценой больше 1000															
<	Меньше	Количество <10 записи в количестве менее 10															
>=	Больше или равно	Цена >=1000 записи с ценой больше или равной 1000															
<=	Меньше или равно	Количество <=10 записи в количестве менее или равным 10															
AND	Логическое И Таблица истинности: <table><tr><td>A</td><td>B</td><td>A AND B</td></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table>	A	B	A AND B	1	1	1	1	0	0	0	1	0	0	0	0	Цена>=100 AND Цена <=500 записи с ценой от 100 до 500 включительно
A	B	A AND B															
1	1	1															
1	0	0															
0	1	0															
0	0	0															
OR	Логическое ИЛИ Таблица истинности: <table><tr><td>A</td><td>B</td><td>A OR B</td></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table>	A	B	A OR B	1	1	1	1	0	1	0	1	1	0	0	0	Тип = 'Овощи' OR Тип = 'Фрукты' записи с типом фрукты или овощи
A	B	A OR B															
1	1	1															
1	0	1															
0	1	1															
0	0	0															
NOT	Реверс значения логического выражения Таблица истинности: <table><tr><td>A</td><td>NOT A</td></tr><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td></tr></table>	A	NOT A	1	0	0	1	Тип NOT 'Фрукты' типы кроме фруктов									
A	NOT A																
1	0																
0	1																
LIKE	Для отбора данных по шаблону. Синтаксис: <выражение>LIKE<шаблон> При этом в шаблоне	Название LIKE '_a%' записи, в названии которых вторая буква a															

	используются обозначения: _ - один символ, % - любое количество символов, [список] - любой символ из списка.	Фамилия LIKE '[А,Б]%' фамилии, начинающиеся на букву А или Б Тип LIKE '[А-Д]%' записи с типом, начинающимся на буквы от А до Д
IN	Позволяет выбрать значения из заданного списка. Синтаксис: <выражение>IN(<выражение>, ;<выражение>, ;...)	Пол IN ('мужской'; 'женский') записи с полом мужской или женский
BETWEEN	Проверяет, находится ли (или нет) выражение в заданном диапазоне. Синтаксис: <выражение>BETWEEN <нижнее значение> AND <верхнее значение>	Цена BETWEEN 1000 AND 12000 записи с ценой от 1000 до 12000 включите
И др.		

2.11.1 Создание ограничения проверки

1. Откройте конструктор таблицы для таблицы, которая будет содержать ограничение.
2. Щелкните правой кнопкой мыши в конструкторе таблицы и выберите **Check Constraints** из контекстного меню.
3. Нажмите кнопку **Add** (Добавить).
4. Введите имя для ограничения в **Constraint name**.
5. В поле выражения (**Expression**) введите выражение типа SQL выражений для проверки ограничения. Например:
Город = 'Томск'
Индекс LIKE '[0-9][0-9][0-9][0-9][0-9]'
Цена >=0
и др.
6. Разверните окно с проверочными ограничениями для установки, когда ограничение обеспечивается:
 - **Check Existing Data on Creation or Enabling** - для проверки ограничения на данные, которые существовали прежде, чем создано ограничение, проверка существующих данных на создание и включение.
 - **Enforce For Replication** - для обеспечения соблюдения ограничений, когда операции репликации происходит на этой таблице.
 - **Enforce for INSERTs and UPDATEs** - для обеспечения соблюдения ограничений, когда строка этой таблицы вставляется или обновляется, проверить Обеспечение для вставок и обновлений.

2.11.2 Удаление проверочного ограничения

1. В меню конструктора таблицы нажмите кнопку **Check Constraints**.
 2. В диалоге **Check Constraints** выберите необходимое ограничение в списке ограничений.
 3. Нажмите кнопку **Delete** (Удалить).
- Ограничение будет удалено из базы данных при сохранении таблицы.

2.12 Определение вычисляемого столбца

1. Выберите столбец, для которого хотите задать формулу вычисляемого столбца.
2. В колонке на вкладке **Properties** (Свойства) в **спецификации вычисляемого столбца** укажите в **Formula** (Формула) выражение для этого столбца.

2.13 Изменение точности типа данных столбцов

Свойство Точность (**Precision**) относится к **decimal** и **numeric** типам.

1. Выберите столбец, для которого вы хотите изменить точность.
2. В колонке на вкладке **Properties** (Свойства) щелкните на ячейке для свойства точности **Precision** и введите значение для точности.

2.14 Установка сортировки для колонки

1. Выберите столбец, для которого хотите установить параметры сортировки (**collation**).
2. Нажмите **Collation** и нажмите кнопку с многоточием (...) справа от свойства. Появится диалоговое окно для сортировки (**Collation dialog box**).
3. Выберите настройки параметров сортировки для выбранного столбца в диалоговом окне Параметры сортировки.

2.15 Установка связей

Связи можно устанавливать различными способами.

2.15.1 Создание внешнего ключа в конструкторе таблиц

1. Откройте конструктор таблицы, в которой будет находиться внешний ключ.

2. В меню конструктора таблицы нажмите кнопку связей/отношений (**Relationships**) или в конструкторе таблиц щелкните на кнопке **Relationships** панели инструментов.
3. В диалоговом окне **Foreign Key Relationships** нажмите кнопку **Add** (Добавить).
4. Нажмите на связь в этом списке связей.
5. Нажмите **Tables and Columns Specification** и нажмите кнопку с многоточием (...) справа от свойства.
6. В диалоге **Tables and Columns** в раскрывающемся списке выберите родительскую таблицу в **Primary Key Table** в связи.
7. Затем укажите дочернюю таблицу из списка в **Foreign key table**.
8. Укажите соответствующие поля для связи из выпадающего списка в диалоге **Tables and Columns**.
9. Укажите имя для связи.
10. Выберите **OK**, чтобы создать связь.

2.15.2 Диалог внешнего ключа (Foreign Key Relationships)

Используйте это окно для создания или изменения связей между таблицами в базе данных.

Внешний ключ может ссылаться на ограничение первичного ключа или ограничение уникальности в другой таблице.

Сложности, создаваемые ссылочной целостностью, сводятся к недопустимости удаления или модификации строки родительской таблицы до тех пор, пока существуют строки дочерней таблицы, ссылающиеся на нее. Если последние остаются после удаления строки родительской таблицы, то нарушаются требования ссылочной целостности. Решение этой проблемы сводится к модификации строк дочерней таблицы как части транзакции, изменяющей строки родительской таблицы.

Чтобы получить доступ к диалоговому окну внешнего ключа, откройте конструктор для таблицы с внешним ключом, щелкните правой кнопкой мыши сетку определения таблицы и нажмите кнопку связей (**Relationships**).

В данном диалоге:

- **Add** - создание новых связей.
- **Delete** - удаляет выбранную связь в списке связей.
- **Check Existing Data on Creation or Re-Enabling** - проверить имеющиеся данные на создание или повторное включение.
- **Tables and Columns Specifications** – показывает какие таблицы и по каким столбцам связаны, что выступает в качестве внешнего ключа и первичного (или уникального) ключа в связях. Чтобы изменить или определить эти значения, нажмите кнопку с многоточием (...) справа от свойства.
- **Foreign Key Base Table** - внешнего ключа таблица. Показывает, какая таблица содержит столбец, выступающий в качестве внешнего ключа в выбранной связи.

- **Foreign Key Columns** - столбцы внешнего ключа. Показывает, какой столбец действует как внешний ключ в выбранной связи.

- **Primary/Unique Key Base Table** - первичный / уникальный ключ родительской таблицы. Показывает, какая таблица содержит столбец, выступающий в качестве первичного (или уникального) ключа в выбранной связи.

- **Primary/Unique Key Columns** - показывает, какой столбец действует как первичный (или уникальный) ключ в выбранной связи.

- **Name** (Имя) - показывает имя связи.

- **Description** (Описание) – показывает описание связей.

- **Enforce for Replications** - поддерживать репликации, если таблицы реплицируются в другой базе данных.

- **Enforce Foreign Key Constraint** - обеспечить ограничения внешнего ключа.

- **INSERT and UPDATE Specification** - расширение для отображения информации для правил удаления и правил обновлений.

- **Delete Rule** – правило на удаление.

- **Update Rule** – правило на обновление.

Для строк дочерней таблицы возможны четыре варианта действий, выбираемых в параметрах **Delete Rule** и **Update Rule** диалога **Foreign Key Relationships**:

- **No Action** - строки таблицы не изменяются. Их присутствие будет предохранять строки родительской таблицы от удаления и модификации. Используйте этот вариант, когда дочерние строки сопровождают значения родительских строк. Вы не сможете изменить или удалить родительские строки, если существуют связанные с ними дочерние.

- **Cascade** - операции удаления и модификации, выполняемые на родительских строках, выполняются также и на дочерних. Используйте этот вариант, когда дочерние данные теряют смысл в отсутствие родительских. Используются механизмы **каскадного удаления** и/или **каскадного обновления** соответственно.

- **Set Null** - этот вариант оставляет дочерние строки нетронутыми, однако изменяет значение внешнего ключа на NULL (разумеется, предполагается допустимость наличия NULL значений). Используйте этот вариант, если хотите разрешить удаление родительских строк без влияния на существование дочерних.

- **Set Default** – родительские строки могут удаляться и изменяться, при этом значения внешнего ключа изменяются на значения по умолчанию, заданные для данного столбца. Этот вариант аналогичен варианту **Set Null**, за исключением того, что на этот раз внешнему ключу присваивается определенное значение.

2.15.3 Удаление ограничения внешнего ключа

1. В меню конструктора таблицы нажмите кнопку связей/отношений **Relationships**.
2. В диалоге **Foreign Key Relationships** выберите связь в этом списке связей.
3. Нажмите кнопку **Delete** (Удалить).

2.16 Диаграмма базы данных (Database Diagram)

Создание диаграмм баз данных можно проводить с помощью инструмента **Database Diagram**, который берет за основу структуры таблиц, созданные в узле **Table**, создает графическое представление таблиц и связей между ними.

Этот инструмент имеет собственный узел в каждой из баз данных. Каждая база данных может иметь множество диаграмм, так как в каждой из них может быть представлен ее конкретный модуль. Вы можете создать одну или несколько диаграмм, иллюстрирующих некоторые или все таблицы, столбцы, ключи и связи в ней. Каждая таблица базы данных может появиться на разном количестве диаграмм. Таким образом, можно создавать различные диаграммы для визуализации различных частей базы данных или, чтобы подчеркнуть различные аспекты дизайна. Это облегчает работу с крупными базами данных.

2.16.1 Создание новой диаграммы базы данных

1. В обозревателе объектов щелкните правой кнопкой мыши узел **Диаграммы базы данных (Database Diagrams)** или диаграмму в этом узле.
2. Выберите **Создать диаграмму базы данных** в контекстном меню. Появится окно **Add Table** (Добавление таблицы).
3. Выберите необходимые таблицы в списке таблиц и нажмите кнопку **Добавить (Add)**.

Таблицы графически отображаются в новой диаграмме базы данных.

Вы можете продолжать добавлять или удалять таблицы, изменять существующие таблицы и изменять связи между таблицами.

После сохранения диаграммы базы данных вы можете сохранить все изменения, внесенные в нее, в том числе изменения, внесенные в таблицы, столбцы и другие объекты базы данных.

2.16.2 Сохранение диаграммы базы данных

1. В меню **File** (Файл) выберите **Save** (Сохранить).
2. Если это новая схема, введите имя диаграммы.
Если вы внесли изменения в существующей диаграмме, в диалоговом окне Сохранить появляется список изменений, которые будут сохранены в базе данных при сохранении схемы.
3. Нажмите **Yes** (или **OK** если это новая диаграмма) для обновления базы данных по диаграмме.

2.16.3 Открытие диаграммы базы данных

1. Раскройте узел Диаграммы базы данных (**Database Diagrams**).
2. Дважды щелкните название диаграммы базы данных, которую требуется открыть.
Или щелкните правой кнопкой мыши имя диаграммы базы данных, которую требуется открыть, а затем выберите **Изменить**.
Диаграмма откроется в конструкторе, из которого вы можете редактировать диаграммы.

2.16.4 Удаление диаграммы базы данных

1. Раскройте узел Диаграммы базы данных (**Database Diagrams**).
2. Щелкните правой кнопкой мыши диаграмму базы данных, которую хотите удалить.
3. Выберите команду Удалить (**Delete**) в контекстном меню.
4. Сообщение попросит вас подтвердить удаление. Выберите **Да**.

2.16.5 Добавление таблицы в существующую диаграмму

1. Убедитесь, что вы подключены к диаграмме базы данных, которую необходимо изменить.
2. Выберите таблицу в папке Таблицы (**Tables**).
3. Перетащите таблицу в диаграмму базы данных, удерживая левую кнопку мыши.
4. Отпустите кнопку мыши.
или
Щелкните правой кнопкой мыши в конструкторе диаграммы и выберите **Добавить таблицу**.

2.16.7 Удаление таблицы из диаграммы базы данных

1. В диаграмме базы данных выберите таблицу, которую необходимо удалить.

- Щелкните правой кнопкой мыши таблицу и выберите **Удалить таблицу из диаграммы (Remove Table from Diagram)** из контекстного меню.

Или нажмите клавишу **ESC**.

Таблицы удаляются из диаграммы, но они продолжают существовать в базе данных.

2.16.8 Проведение связей в диаграмме базы данных

- В диаграмму базы данных добавьте таблицы, которые необходимо связать.
- Свяжите ключевое (уникальное) поле родительской таблицы с полем внешнего ключа дочерней таблицы путем перетаскивания, удерживая левую кнопку мыши.
- В появившемся диалоговом окне **Tables and Columns** проверьте правильность установки связи.
- Нажмите **ОК**, если связь устанавливается верно.
- В появившемся диалоговом окне **Foreign Key Relationships** проверьте правильность установленной связи, установите, если необходимо, параметры в данном окне.
- Нажмите **ОК**, если связь устанавливается верно.

3 Задание на лабораторную работу №1

Создайте базу данных для своей предметной области в выбранной СУБД.

Лабораторная работа выполняется по результатам разработанной структуры БД в индивидуальном задании согласно варианту.

Требования к базе данных (БД):

- БД должна быть реляционной;
- БД должна быть нормализована по 3НФ (или БКНФ);
- БД должна состоять минимум из 3-4 связанных таблиц (на каждого студента);
- в таблицах БД должны быть наложены ограничения на поля таблиц (по усмотрению разработчиков решается, на какие поля будут наложены ограничения). Должно быть не менее 3 ограничений на разные поля таблиц (без учета ограничений PRIMARY KEY, FOREIGN KEY). Ограничения проверки, значения по умолчанию обязательно;
- должна быть обеспечена ссылочная целостность;
- в БД должны быть обязательно поля разных типов данных, в том числе поля:
 - позволяющие хранить рисунки;
 - позволяющие хранить длинный текст;
 - значения в которых должны соответствовать списку значений, например Пол мужской или женский;
 - значения логического типа.

Ниже приведен пример создания БД для кафе (описание БД приведено в разделе 4) под управлением СУБД Microsoft SQL Server (раздел 5).

4 Описание базы данных для кафе

Создадим следующую структуру базы данных (рисунок 4.1) для кафе.

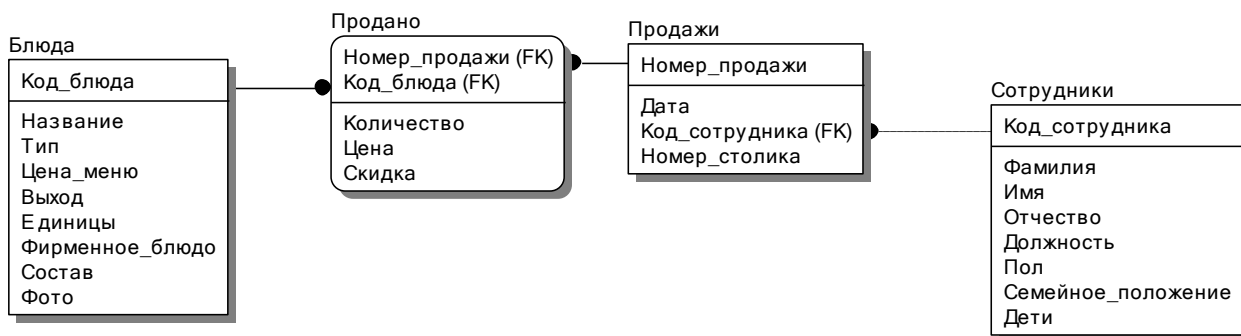


Рисунок 4.1 – Структура БД для кафе

4.1 Описание таблицы Блюда

Имя поля	Тип данных поля	Описание	Ограничения
Код_блюда	Числовой	Код блюда в кафе. Уникальный идентификатор блюда.	Первичный ключ. Условие на значение: ≥ 0
Название	Текстовый	Название блюда.	Размер поля 40. Обязательное поле.
Тип	Текстовый	Тип блюда (салаты, десерты, супы и др.).	Размер поля 30. Обязательное поле.
Цена_меню	Денежный	Цена блюда по меню.	Обязательное поле. Значение по умолчанию 0. Условие на значение: ≥ 0
Выход	Числовой	Количество готового блюда в соответствующих единицах.	Значение по умолчанию 0. Условие на значение: ≥ 0
Единицы	Текстовый	Единицы измерения	Размер поля 5. Допустимы значения "гр." или "мл." или "шт." Значение по умолчанию "гр."

Фирменное_блюдо	Логический	Указывает является ли данное блюдо фирменным	Да/Нет
Состав	Длинный текст	Описание состава блюда.	
Фото	Фотография	Фотография готового блюда.	

Примечание:

Фирменное блюдо — это особенное блюдо, которое готовится шеф-поваром. Это такое блюдо, которое нельзя попробовать в других кафе. Фирменное блюдо может представлять собой горячее, горячую или холодную закуску, салат, десерт или напиток.

4.2 Описание таблицы Сотрудники

Имя поля	Тип данных поля	Описание	Ограничения
Код_сотрудника	Счетчик	Уникальный идентификатор сотрудника.	Первичный ключ.
Фамилия	Текстовый	Фамилия сотрудника	Размер поля 30. Обязательное поле.
Имя	Текстовый	Имя сотрудника	Размер поля 30. Обязательное поле.
Отчество	Текстовый	Отчество сотрудника	Размер поля 30.
Должность	Текстовый	Должность сотрудника	Размер поля 30. Обязательное поле.
Пол	Текстовый	Пол сотрудника	Размер поля 10. Условие на значение:

			‘мужской’ или ‘женский’
Семейное_положение	Текстовый	Семейное положение	Размер поля 15. Условие на значение: ‘Женат’ или ‘Замужем’ или ‘Холост’ или ‘Не замужем’
Дети	Логический	Имеются ли дети у сотрудника	Да/Нет

4.3 Описание таблицы Продажи

Имя поля	Тип данных поля	Описание	Ограничения
Номер_продажи	Счетчик	Уникальный идентификатор продажи.	Первичный ключ.
Дата	Дата/время	Дата продажи	
Код_сотрудника	Числовой	Идентификатор сотрудника, который оформил продажу. Ссылается на таблицу Сотрудники.	Внешний ключ. Обязательное поле.
Номер_столика	Текстовый	Номер столика в кафе	Размер поля 10.

4.4 Описание таблицы Продано

Имя поля	Тип данных поля	Описание	Ограничения
Номер_продажи	Числовой	Номер продажи. Ссылается на таблицу Продажи.	Входит в состав первичного ключа. Внешний ключ.
Код_блюда	Числовой	Код блюда. Ссылается на таблицу Блюда.	Входит в состав первичного ключа. Внешний ключ.
Количество	Числовой	Количество проданных блюд	Целое число. Обязательное поле.

			Значение по умолчанию 1. Условие на значение: >0
Цена	Денежный	Цена блюда на момент продажи	Обязательное поле. Значение по умолчанию: 0 Условие на значение: >=0
Скидка	Числовой	Скидка на конкретное блюдо в продажи	Значение по умолчанию 0. Условие на значение: >=0 и <=1

Примечание:

В лабораторной работе №1 для удобства объяснения и понимания предметной области используются названия таблиц и полей на русском языке.

5 Создание БД под управлением СУБД Microsoft SQL Server для кафе

В данной работе описываются возможности среды SQL Server Management Studio  Среда SQL Server Management Studio для реализации проекта учебной базы данных под управлением СУБД Microsoft SQL Server .

В среде SQL Server Management Studio графические средства сочетаются с набором функциональных редакторов скриптов для доступа разработчиков и администраторов с разным уровнем знаний к SQL Server.

Главным инструментом SQL Server Management Studio является **Object Explorer (Обозреватель объектов)**, который позволяет пользователю просматривать, извлекать объекты сервера, а также ими управлять.

Для использования обозревателя объектов следует сначала подключиться к серверу. На панели инструментов обозревателя объектов нажмите кнопку **Соединить** и выберите тип сервера из раскрывающегося списка. Открывается диалоговое окно **Соединение с сервером**. Для подключения следует указать, по крайней мере, **имя сервера и правильные данные для проверки подлинности**.

При подключении к серверу можно указать дополнительные данные соединения в диалоговом окне Соединение с сервером. В диалоговом окне Соединение с сервером сохраняются использованные в последний раз настройки, которые будут использоваться для новых соединений, например для открытия новых окон редактора кода.

Чтобы задать выборочные настройки соединения, выполните следующее.

1. На панели инструментов обозревателя объектов нажмите кнопку **Соединить**, а затем выберите тип сервера, к которому производится соединение. Откроется диалоговое окно Соединение с сервером.
2. В поле **Имя сервера** введите имя экземпляра SQL Server.
3. Нажмите **Параметры**. В диалоговом окне Соединение с сервером будут показаны дополнительные **параметры**.
4. Набор доступных параметров зависит от типа сервера.

Для компонента **Database Engine** доступны следующие параметры.

Параметр	Описание
Подключиться к базе данных	Позволяет выбрать одну из доступных баз данных на сервере. Список содержит только те базы данных, для просмотра которых имеется разрешение.
Сетевой протокол	Позволяет выбрать один из вариантов: «Общая память»,

«TCP/IP» или «Именованные каналы».

Размер сетевого пакета	Задается в байтах. Значение по умолчанию равно 4 096 байтам.
Время ожидания соединения	Задается в секундах. Значение по умолчанию равно 15 секундам.
Время ожидания выполнения	Задается в секундах. Значение по умолчанию (0) означает, что выполнение никогда не будет прервано из-за превышения времени ожидания.
Шифрование соединения	Задаёт принудительное шифрование.

Чтобы добавить заданный сервер в список зарегистрированных серверов, перейдите на вкладку Зарегистрированный сервер, затем щелкните позицию, где следует поместить сервер, и завершите соединение.

В среде SQL Server Management Studio имеется множество средств для этапов разработки и администрирования. Некоторые средства можно использовать для любого компонента SQL Server, другие — только для определенных компонентов.

Обозреватель объектов может быть использован для всех компонентов SQL Server.

Инструмент	Назначение
Обозреватель объектов	Обзор серверов, создание и размещение объектов, управление источниками данных и просмотр журналов. Доступ к этому средству осуществляется из меню Вид.

В лабораторной работе используется СУБД **Microsoft® SQL Server® 2012 Express (Microsoft SQL Server 2012 Express LocalDB)** — мощная и надежная бесплатная система управления данными, обеспечивающая функциональное и надежное хранилище данных для веб-сайтов и настольных приложений.

SQL Server Express LocalDB — это упрощенная версия SQL Server, которая обладает множеством возможностей по программированию баз данных, аналогичных SQL Server.

SQL Server Express LocalDB выполняется в пользовательском режиме, и его можно установить более быстро, с меньшим числом требований и без конфигурации.

В следующей таблице описаны различия между SQL Server Express LocalDB и SQL Server Express.

	SQL Server Express LocalDB	SQL Server Express
Тип базы данных при создании базы данных, основанной на службах	В Visual Studio 2012 и Visual Studio 2013 — SQL Server Express LocalDB	В Visual Studio 2010 и более ранних версиях — SQL Server Express
Имя экземпляра SQL Server в средствах и параметрах	(LocalDB)\v11.0	SQLEXPRESS
Значение источника данных в строке подключения	(LocalDB)\v11.0	.\SQLEXPRESS
Значение AttachDbFilename в строке подключения	путь к файлу	путь к файлу
Требуется пользовательский экземпляр ("User Instance=True" в строке подключения)	Нет	Да
Расширение файла базы данных	MDF	MDF

Преимущества SQL Server Express LocalDB:

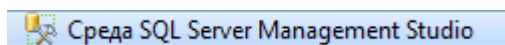
- SQL Server Express LocalDB совместима с основанными на службах выпусками SQL Server для функций, включаемых SQL Server Express LocalDB. В SQL Server можно переместить все базы данных или код Transact-SQL из SQL Server Express LocalDB в SQL Server или SQL Azure, не выполняя никаких действий по обновлению. Поэтому можно использовать SQL Server Express LocalDB для разработки приложений, ориентированных на все выпуски SQL Server.
- SQL Server Express LocalDB поддерживает те же оптимизатор запросов и средство обработки запросов, что и более высокие выпуски SQL Server.

5.1 Создание БД для кафе

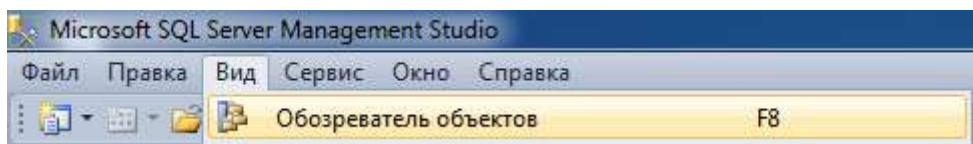
Создадим базу данных, структура и описание которой приведены в разделе 4.

Создайте для своей БД отдельную папку, например: C:\DB_Cafe. В данную папку будем в дальнейшем сохранять свою БД.

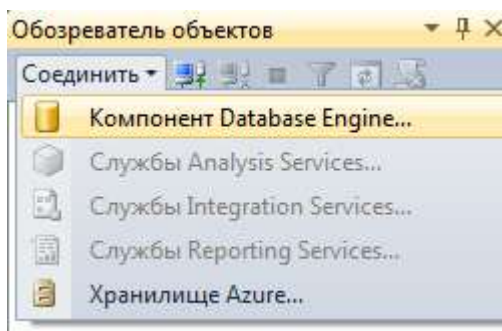
Запустите среду SQL Server Management Studio:



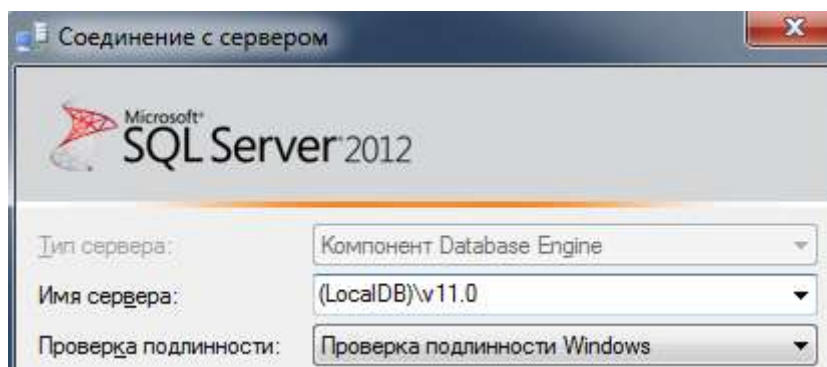
Выберете меню **Вид (View)**, в нем **Обозреватель объектов (Object Explorer)** или нажмите F8:



В окне **Обозреватель объектов (Object Explorer)** выберете **Компонент Database Engine**:



В диалоговом окне **Соединение с сервером** укажите:



Microsoft SQL Server предоставляет средства разделенного управления объектами сервера. Выберите необходимый тип аутентификации:

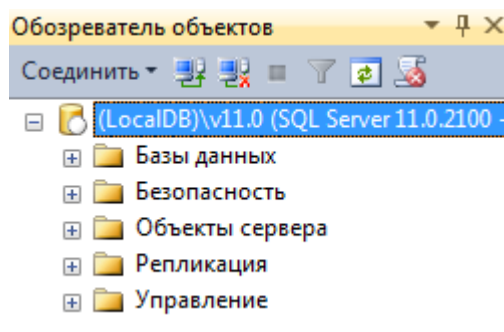
- режим аутентификации Windows (**Windows Authentication**). При установке данный режим предлагается по умолчанию. В этом случае аутентификация пользователя осуществляется операционной системой Windows. Затем SQL Server использует аутентификацию операционной системы для определения уровня доступа;

- регистрационные записи SQL Server (**SQL Server Authentication**) - учетные записи создаются и управляются в самой СУБД.

Использование для доступа к Microsoft SQL Server обоих методов называют **смешанным режимом**. Смешанный режим позволяет проводить аутентификацию пользователя как средствами операционной системы, так и с применением учетных записей Microsoft SQL Server.

Нажмите кнопку **Соединить**.

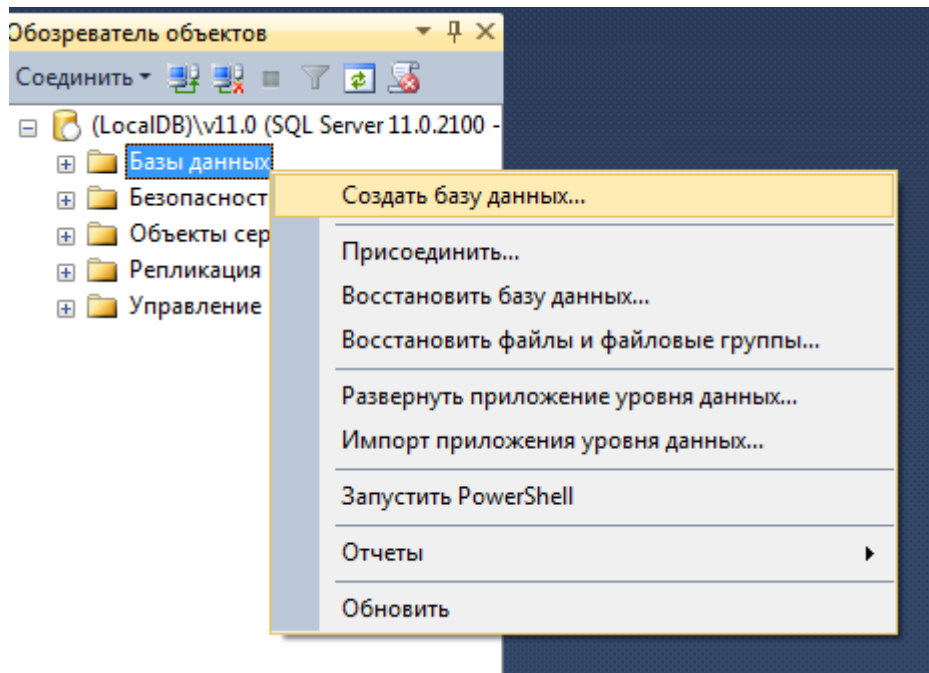
В результате получаем:



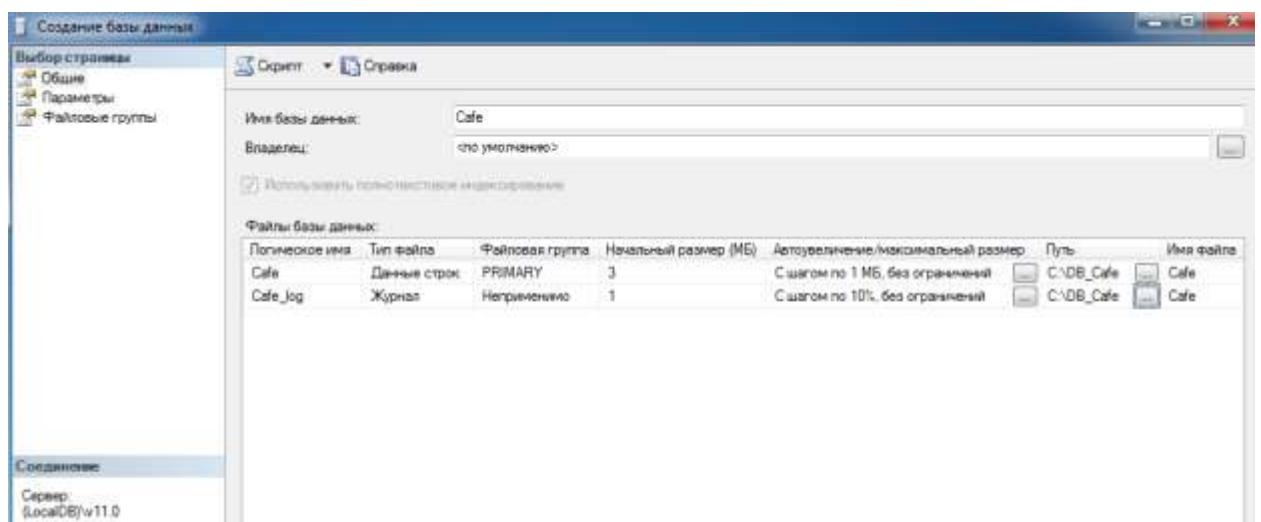
Примечание:

Предварительно у Вас должен быть доступ к SQL Server с правами на создание БД.

На узле **Базы данных** в обозревателе объектов нажмите правую кнопку мыши и выберете **Создать базу данных**:



В диалоговом окне укажите для новой базы данных имя **Cafe** и необходимый путь к БД:



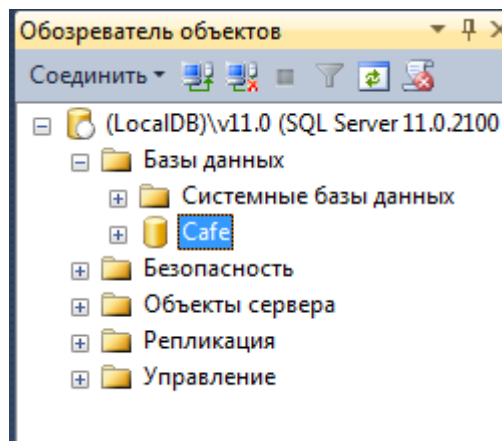
Нажмите ОК.

База данных состоит из двух файлов (точнее, из двух наборов файлов):

- файл данных (данные строк) с заданным именем и расширением **Cafe.mdf**;
- файл журнала транзакций с расширением **Cafe_log.ldf**.



В результате получаем:



При создании БД необходимо указать **параметры сортировки** для поддержки необходимого языка.

Параметры сортировки определяют правила, используемые при сортировке и сравнении данных.

Параметры сортировки могут учитывать регистр, диакритические знаки, тип японской азбуки, ширину символов.

Эти параметры задаются путем их добавления к имени параметров сортировки.

Например, параметр `Japanese_Bushu_Kakusu_100_CS_AS_KS_WS` определяет параметры сортировки с учетом диакритических знаков, регистра, типа японской азбуки и ширины символов. В следующей таблице описывается режим работы, связанный с этими параметрами.

Параметр	Описание
С учетом регистра (_CS)	Различаются буквы верхнего и нижнего регистров. При выборе этого параметра буквы нижнего регистра при сортировке ставятся перед соответствующими буквами верхнего регистра. Если этот параметр не установлен, параметры сортировки не будут учитывать регистр. То есть при сортировке SQL Server считает буквы верхнего и нижнего регистров идентичными друг другу. Можно явно выбрать нечувствительность к регистру , указав параметр _CI .
С учетом диакритических знаков (_AS)	Различаются символы с диакритическими знаками и без них. Например, «а» отлично от «á». Если этот параметр не установлен, параметры сортировки не будут учитывать диакритические знаки. То есть, при сортировке SQL Server считает буквы с диакритическими знаками и без них идентичными друг другу. Можно явно выбрать нечувствительность к диакритическим знакам , указав параметр _AI .

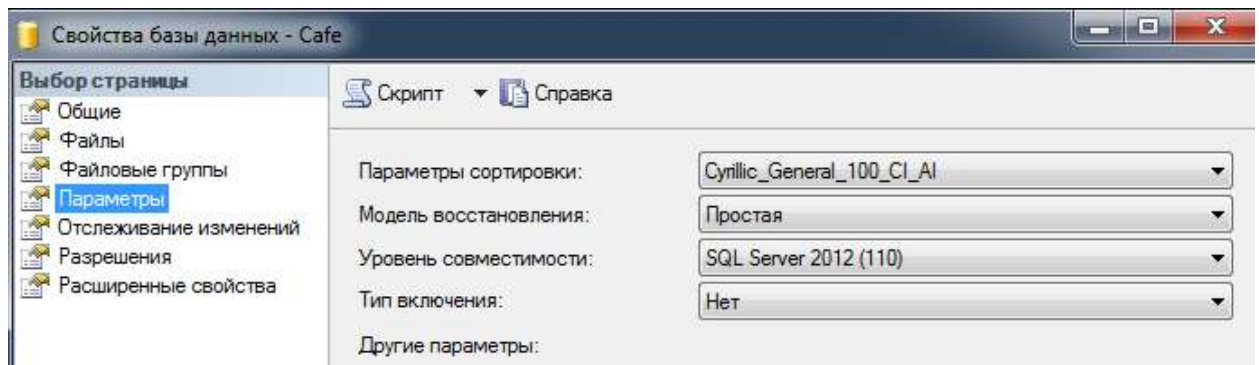
С учетом типа японской азбуки (_KS)

Различаются два вида японской азбуки: Хирагана и Катакана. Если этот параметр не выбран, параметры сортировки не будут учитывать тип японской азбуки. То есть, при сортировке SQL Server рассматривает символы хирагана и катакана как идентичные. Пропуск этого параметра является единственным способом указания нечувствительности к типу японской азбуки.

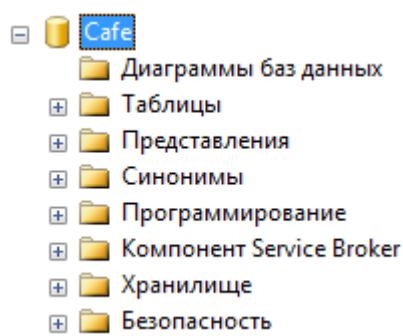
С учетом ширины символов (_WS)

Различаются одинаковые символы, представленные одним и двумя байтами. Если этот параметр не выбран, при сортировке SQL Server рассматривает однобайтовые и двухбайтовые представления одних и тех же символов как идентичные. Пропуск этого параметра является единственным способом указания нечувствительности к ширине символов.

Например, выбор для кириллицы Cyrillic_General_100_CI_AI:



После создания БД в новой базе данных станут доступны ее объекты:



Примечание:

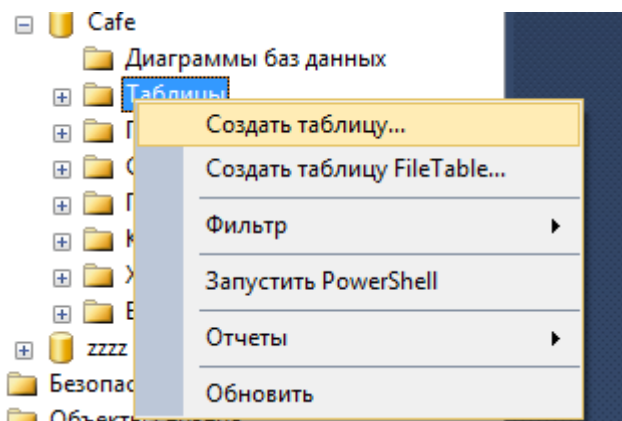
Целью данной лабораторной работы является создание таблиц базы данных, установка межтабличных связей, наложение ограничений на данные в таблицах БД. Остальные объекты БД пока не рассматриваются.

5.2 Создание таблиц в БД

Создайте таблицы «Блюда», «Сотрудники», «Продажи», «Продано» согласно описанию в разделе 4.

5.2.1 Таблица «Блюда»

В нашей БД **Cafe** выделите узел Таблицы (**Tables**), нажмите на нем правую кнопку мыши и выберите пункт для создания новой таблицы БД:



После этого откроется конструктор таблицы.

В появившемся окне **конструктора таблицы** введите соответствующие колонки:

- **Имя столбца (Column Name)** – имя соответствующего столбца;
- **Тип данных (Data Type)** – необходимый тип данных;
- **Разрешить значения Nulls (Allow Nulls)** – допустимы ли NULL значения в данном поле.

Для таблицы **Блюда** задайте в конструкторе таблицы:

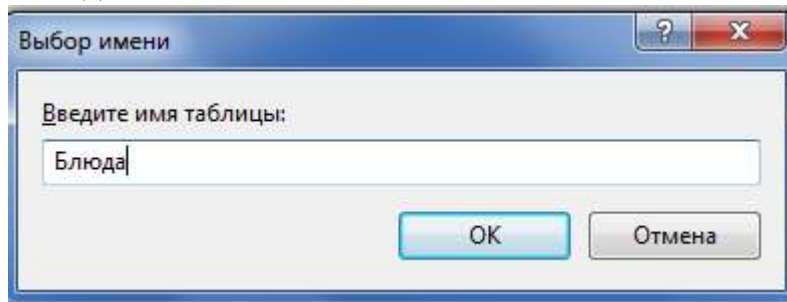
Имя столбца Column Name	Тип данных Data Type	Разрешить значения Nulls Allow Nulls
Код_блюда	int	
Название	nvarchar(40)	
Тип	nvarchar(30)	
Цена_меню	money	
Выход	int	Флаг установлен
Единицы	nvarchar(5)	Флаг установлен
Фирменное_блюдо	bit	Флаг установлен
Состав	nvarchar(MAX)	Флаг установлен
Фото	varbinary(MAX)	Флаг установлен

Сохраните таблицу с именем **Блюда**.

Для этого из меню Файл (**File**) выберете **Сохранить имя таблицы (Save table name)**. В диалоговом окне введите имя таблицы и нажмите кнопку **OK**.

Или нажмите кнопку **Save Table** либо на панели инструментов, либо вызовом нажатия правой кнопки мыши в конструкторе таблицы.

В появившемся диалоговом окне введите имя таблицы. В данном случае **Блюда**:

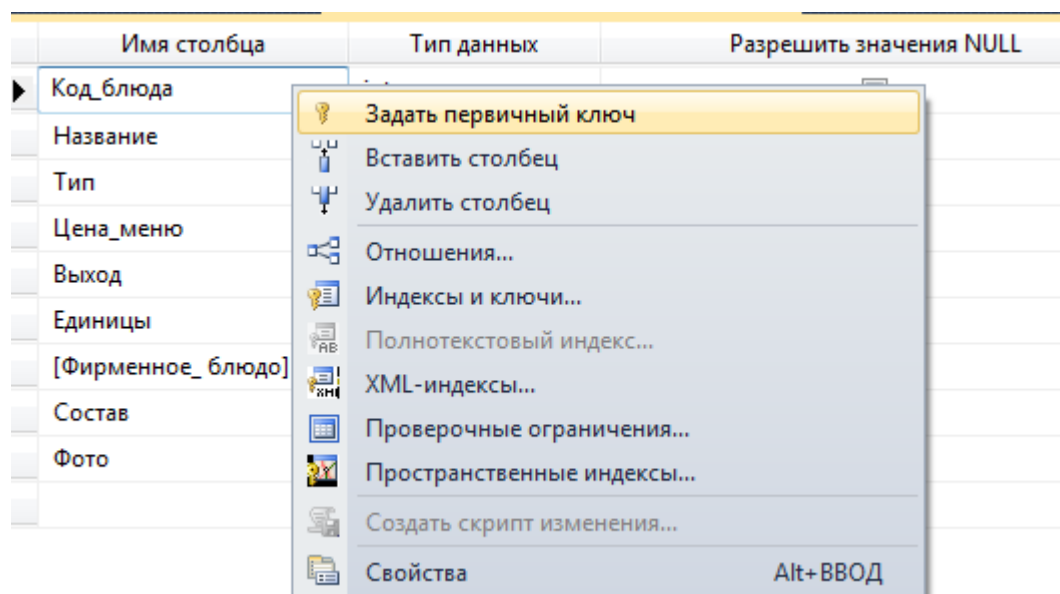


Теперь зададим необходимые ограничения для соответствующих полей.

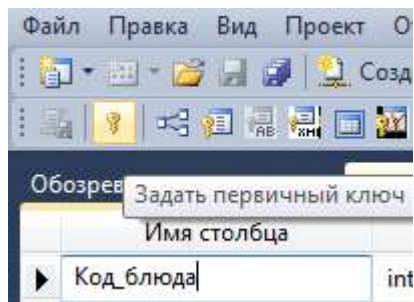
5.2.1.1 Поле Код_блюда

5.2.1.1.1 Установка первичного ключа для поля Код_блюда

Выделите строку для поля Код_блюда, нажмите на ней правую кнопку мыши и выберете **Задать первичный ключ (Set Primary Key)**:



Или нажмите соответствующую кнопку на панели инструментов:



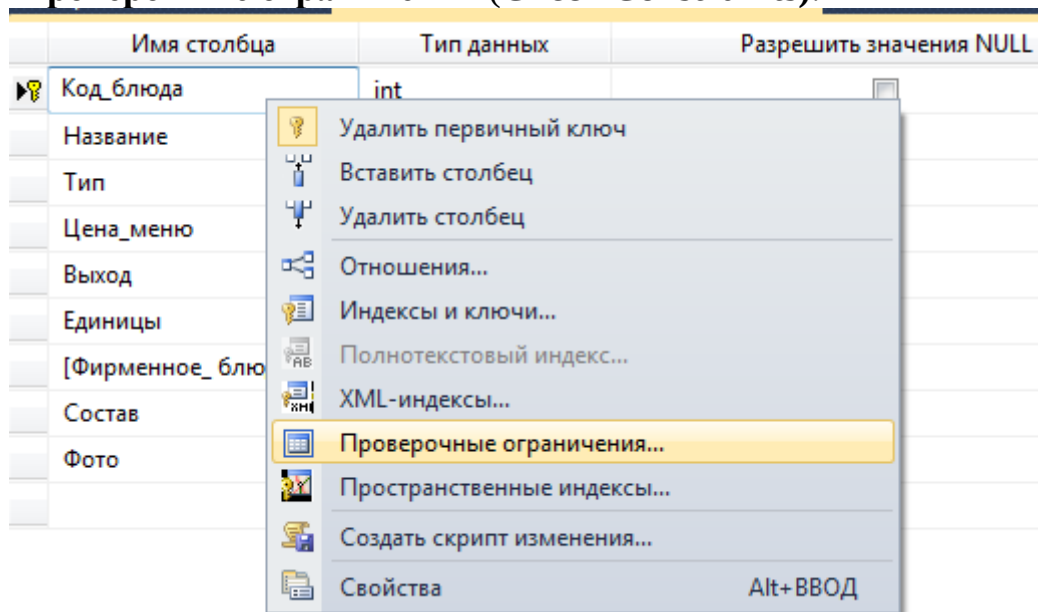
Теперь возле Код_блюда стоит знак первичного ключа:

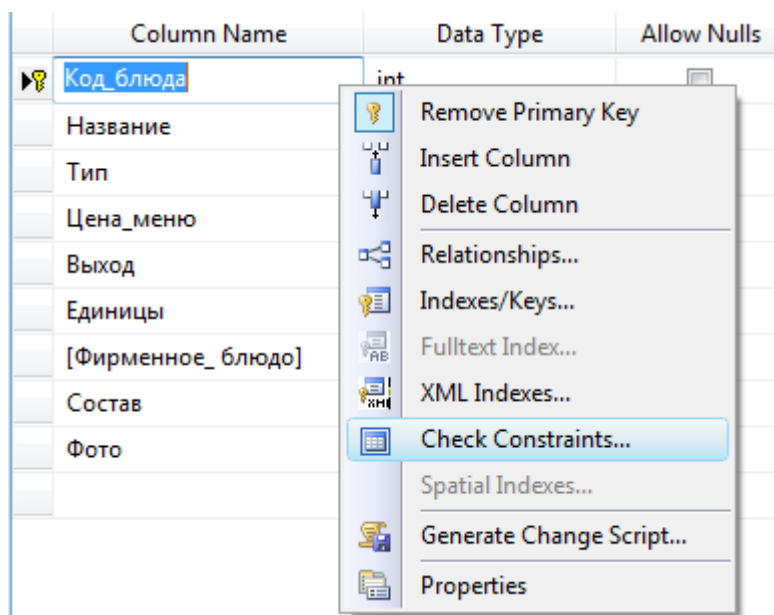
Имя столбца	Тип данных	Разрешить значения NULL
Код_блюда	int	☐

NULL значения для данного поля не допустимы.

5.2.1.1.2 Условие на значение Код_блюда ≥ 0

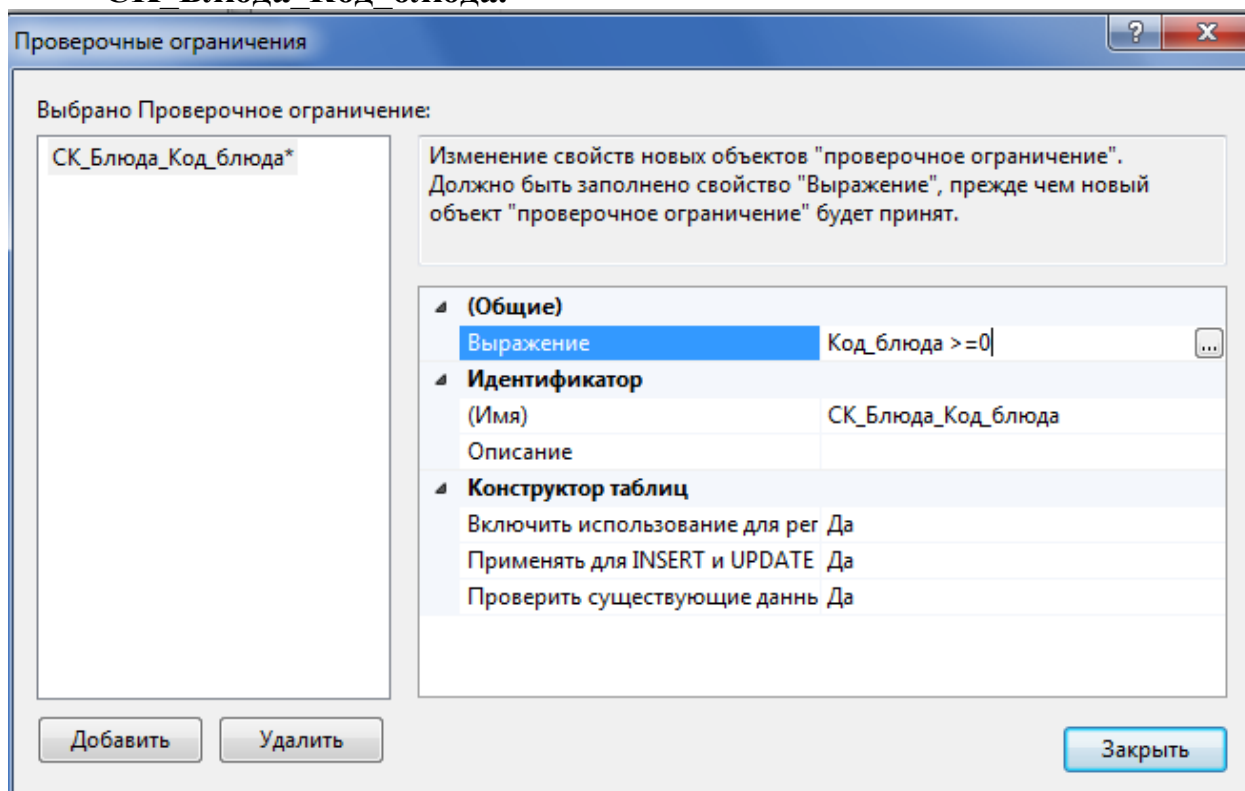
Нажмите правую кнопку мыши в конструкторе таблицы и выберете пункт **Проверочные ограничения (Check Constraints)**:

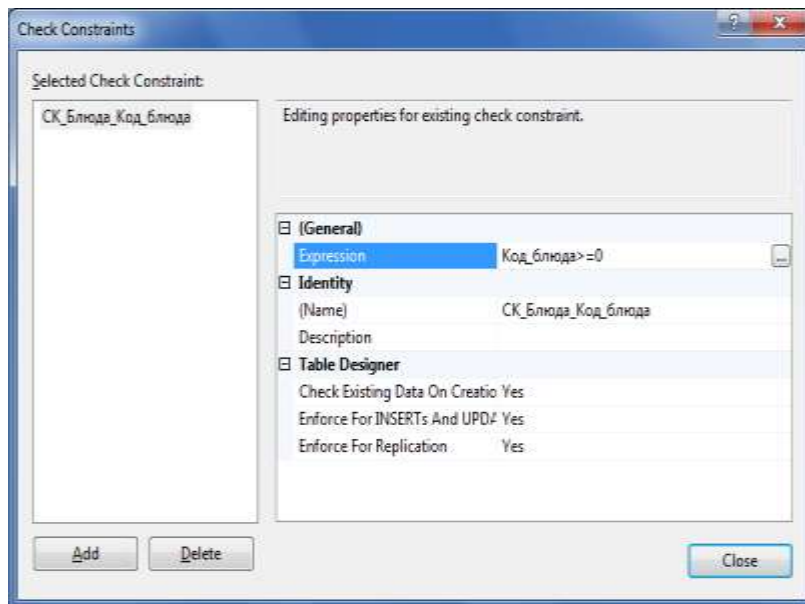




В появившемся диалоговом окне нажмите кнопку **Добавить (Add)**.
 Затем в строке **Выражение (Expression)** укажите условие на значения для **Код_блюда** ≥ 0 .

Назовите ограничение в свойстве **Имя (Name)** в **СК Блюда Код блюда**.





Нажмите **Заккрыть (Close)** и перейдите к конструктору таблицы **Блюда**.

5.2.1.2 Поле Название

Размер поля текстовый до **40 символов**. **Обязательное поле** (запрет NULL значений):

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
▶	Название	nvarchar(40)	☐

ИЛИ

	Column Name	Data Type	Allow Nulls
🔑	Код_блюда	int	☐
▶	Название	nvarchar(40)	☐

5.2.1.3 Поле Тип

Размер поля текстовый до **30 символов**. **Обязательное поле** (запрет NULL значений):

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
▶	Тип	nvarchar(30)	☐

5.2.1.4 Поле Цена_меню

5.2.1.4.1 **Тип данных – денежный. Обязательное поле (запрет NULL значений):**

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
▶	Цена_меню	money	☐

5.2.1.4.2 Значение по умолчанию 0

Перейдите в окно свойств поля **Свойства столбца (Column Properties)** для поля **Цена_меню**. В строке **Значения по умолчанию или привязка (Default Value or Binding)** укажите значение 0:

Имя столбца	Тип данных	Разрешить значения NULL
Код_блюда	int	☐
Название	nvarchar(40)	☐
Тип	nvarchar(30)	☐
Цена_меню	money	☐

Свойства столбца

(Общие)

Имя

Цена_меню

Значение по умолчанию или привязка

0

Разрешить значения NULL

Нет

Тип данных

money

Конструктор таблиц

RowGuid

Нет

Детерминированный

Да

Имеет подписчик, отличный от подписчика SQL Server

Нет

Индексируемый

Да

Набор столбцов

Нет

Не для репликации

Нет

Описание

Опубликован слиянием

Нет

Опубликован через службы DTS

Нет

Параметры сортировки

< база данных по умолчанию >

Размер

8

Разряженный

Нет

Реплицировано

Нет

Сжатый тип данных

money

Спецификация вычисляемого столбца

Нет

Спецификация идентификатора

Нет

Спецификация полнотекстового столбца

Нет

ИЛИ

	Column Name	Data Type	Allow Nulls
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
▶	Цена_меню	money	☐
	Выход	int	☒
	Единицы	nvarchar(5)	☒
	[Фирменное_блюдо]	bit	☒
	Состав	nvarchar(MAX)	☒
	Фото	varbinary(MAX)	☒

Column Properties

🔍

⬆

⬇

🔍

⊖ (General)

(Name) Цена_меню

Allow Nulls No

Data Type money

Default Value or Binding 0

⊖ Table Designer

Collation <database default>

⊕ Computed Column Specification

Condensed Data Type money

Description

Deterministic Yes

DTS-published No

⊕ Full-text Specification

Has Non-SQL Server Subscriber No

⊕ Identity Specification

No

Indexable Yes

Default Value or Binding

5.2.1.4.3 Условие на значение Цена_меню >=0

Аналогично как для Код_блюда (раздел 5.2.1.1.2).
Назовите ограничение СК_Блюда_Цена_меню.

Проверочные ограничения

?

✕

Выбрано Проверочное ограничение:

СК_Блюда_Код_блюда

СК_Блюда_Цена_меню

Изменение свойств существующих объектов "проверочное ограничение".

⚡ (Общие)

Выражение Цена_меню >=0

⚡ Идентификатор

(Имя) СК_Блюда_Цена_меню

Описание

⚡ Конструктор таблиц

Включить использование д/ Да

Применять для INSERT и UPI Да

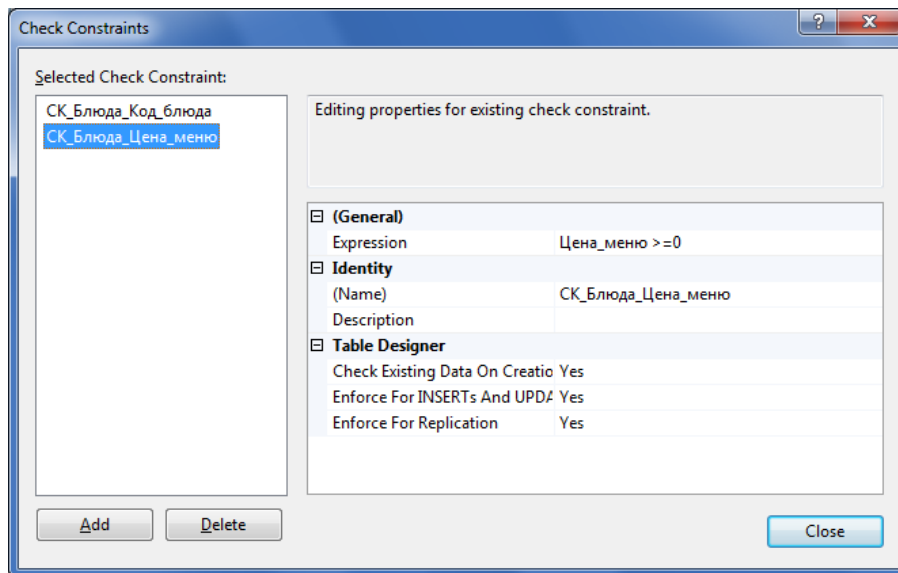
Проверить существующие / Да

Добавить

Удалить

Заккрыть

ИЛИ



5.2.1.5 Поле Выход

5.2.1.5.1 Тип данных – целые числа, NULL значения допустимы:

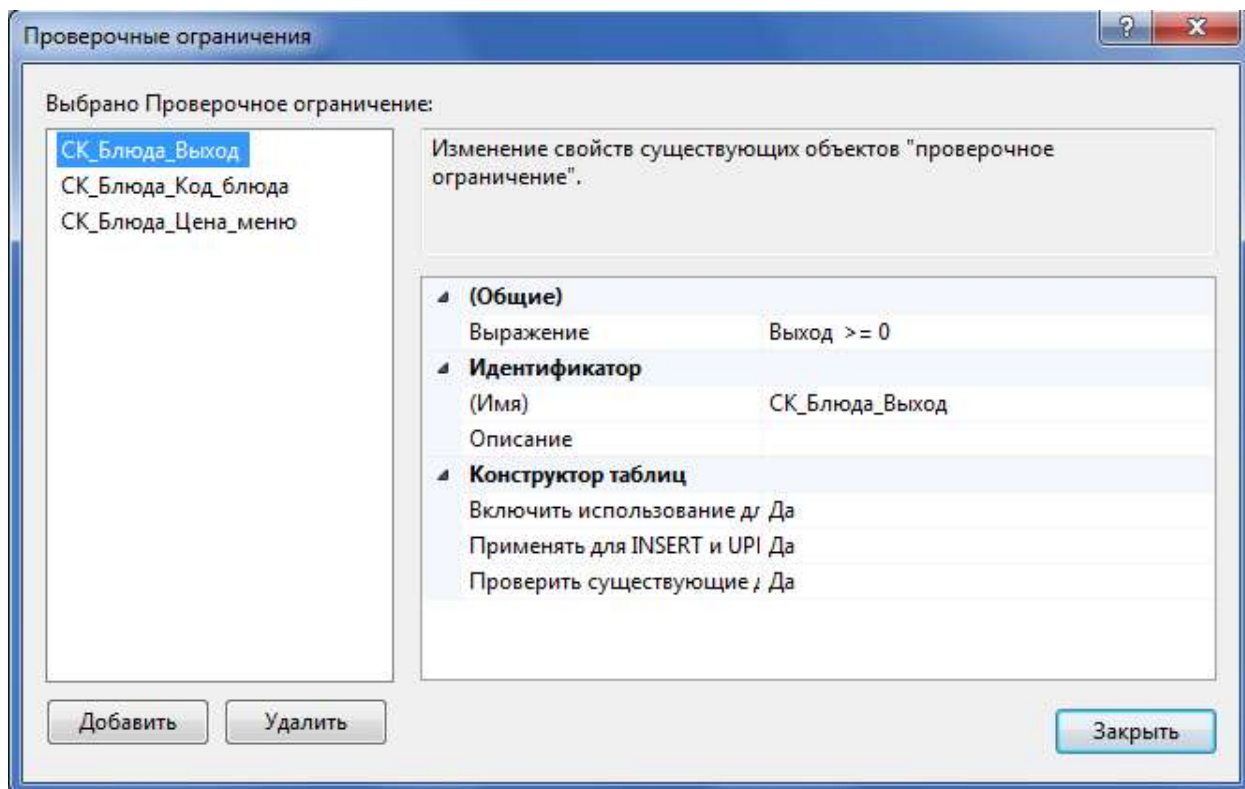
	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
▶	Выход	int	☒

5.2.1.5.2 Значение по умолчанию 0

Аналогично как для поля Цена_меню (раздел 5.2.1.4.2).

5.2.1.5.3 Условие на значение Выход ≥ 0 :

Аналогично как для Код_блюда (раздел 5.2.1.1.2). Назовите ограничение СК_Блюда_Выход.



5.2.1.6 Поле Единицы

5.2.1.6.1 Размер поля **текстовый до 5 символов, NULL значения допустимы:**

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
▶	Единицы	nvarchar(5)	☒

5.2.1.6.2 Значение по умолчанию 'гр.'

Аналогично как для поля Цена_меню (раздел 5.2.1.4.2) только укажите значение 'гр.'

	Имя столбца	Тип данных	Разрешить значения NULL
?	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
▶	Единицы	nvarchar(5)	☒

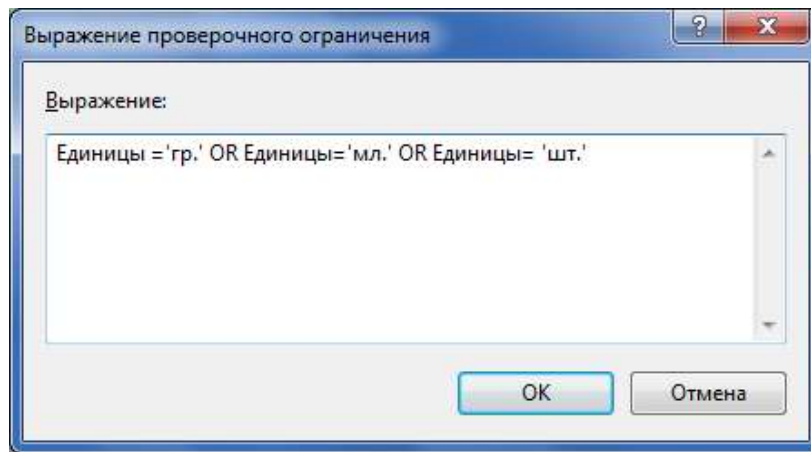
Свойства столбца	
▲ (Общие) (Имя) Единицы Длина 5 Значение по умолчанию или привязка 'гр.' Разрешить значения NULL Да Тип данных nvarchar	

5.2.1.6.3 Допустимы значения 'гр.' или 'мл.' или 'шт.':

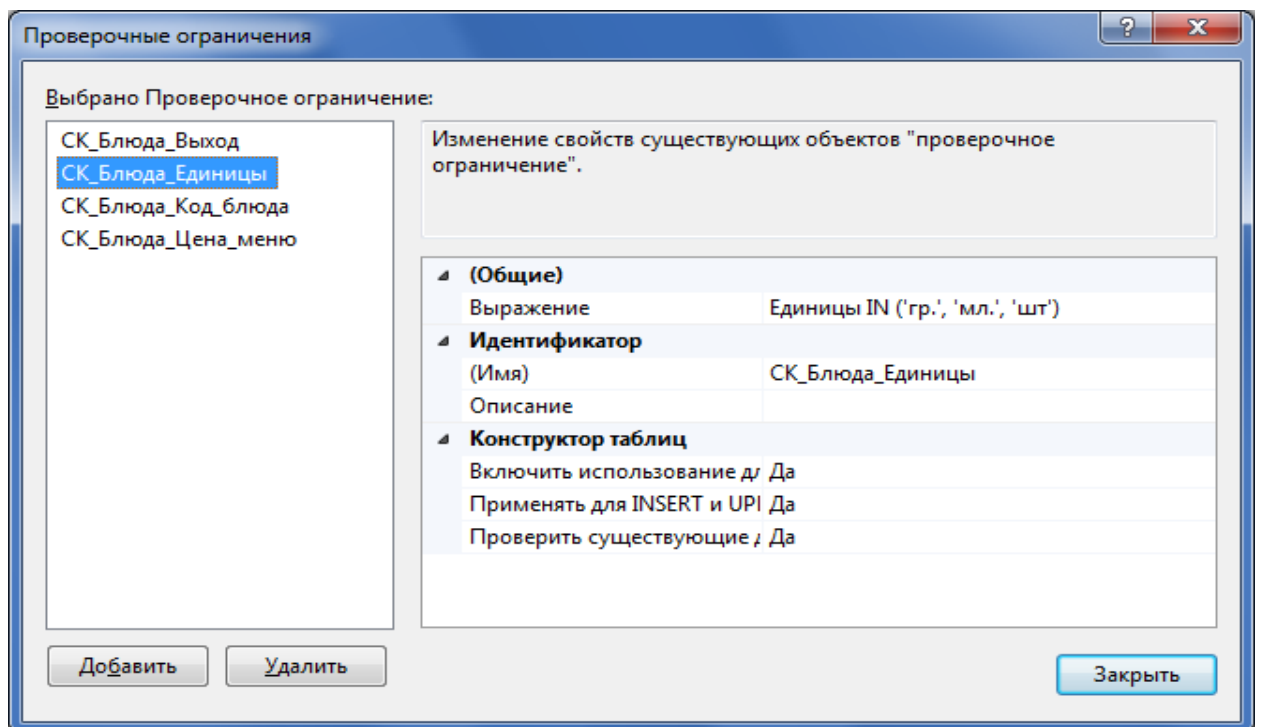
В строке **Выражение (Expression)** укажите условие на значения Единицы IN ('гр.', 'мл.', 'шт'):

ИЛИ

Или Единицы ='гр.' OR Единицы='мл.' OR Единицы= 'шт.'



Назовите ограничение в свойстве **Имя (Name)** в **СК_Блюда_Единицы**.



5.2.1.7 Поле Фирменное_блюдо

Выбран тип данных **bit** - целочисленный тип данных, который может принимать значения 1, 0 или NULL.

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
	Единицы	nvarchar(5)	☒
▶	Фирменное_блюдо	bit	☒

5.2.1.8 Поле Состав

Был выбран тип данных:

nvarchar (max) - символные данные в Юникоде переменной длины. Аргумент **max** указывает, что максимальный размер хранилища равен $2^{31}-1$ байт.

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
	Единицы	nvarchar(5)	☒
	[Фирменное_блюдо]	bit	☒
▶	Состав	nvarchar(MAX)	☒

5.2.1.9 Поле Фото

Был выбран тип данных:

varbinary(max) - двоичные данные переменной длины. **max** означает максимальную длину хранения, которая составляет $2^{31}-1$ байт.

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
	Единицы	nvarchar(5)	☒
	[Фирменное_блюдо]	bit	☒
	Состав	nvarchar(MAX)	☒
▶	Фото	varbinary(MAX)	☒

5.2.1.10 Сохраните таблицу Блюда.

После указания ограничений на поля, сохраните таблицу Блюда.

5.2.1.11 Конструктор таблицы Блюда имеет вид:

	Имя столбца	Тип данных	Разрешить значения NULL
►	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
	Единицы	nvarchar(5)	☒
	[Фирменное_блюдо]	bit	☒
	Состав	nvarchar(MAX)	☒
	Фото	varbinary(MAX)	☒

ИЛИ

	Column Name	Data Type	Allow Nulls
►	Код_блюда	int	☐
	Название	nvarchar(40)	☐
	Тип	nvarchar(30)	☐
	Цена_меню	money	☐
	Выход	int	☒
	Единицы	nvarchar(5)	☒
	[Фирменное_блюдо]	bit	☒
	Состав	nvarchar(MAX)	☒
	Фото	varbinary(MAX)	☒

5.2.2 Таблица «Сотрудники»

Проведите создание таблицы **Сотрудники** аналогичным образом с необходимыми ограничениями.

Для таблицы **Сотрудники** задайте в конструкторе таблицы:

Имя столбца Column Name	Тип данных Data Type	Разрешить значения Nulls Allow Nulls
Код_сотрудника	int	
Фамилия	nvarchar(30)	
Имя	nvarchar(30)	
Отчество	nvarchar(30)	Флаг установлен
Должность	nvarchar(30)	
Пол	nvarchar(10)	Флаг установлен
Семейное_положение	nvarchar(15)	Флаг установлен
Дети	bit	Флаг установлен

Теперь зададим необходимые ограничения для соответствующих полей.

5.2.2.1 Поле Код_сотрудника

5.2.2.1.1 Установка первичного ключа

Данное поле является первичным ключом. Установите первичный ключ по аналогии с разделом 5.2.1.1.1.

5.2.2.1.2 Тип данных счетчик

Тип данных для данного поля решено установить в счетчик (автоинкремент). Значения в данном поле будут указываться, начиная с 1 с шагом 1. Данное поле редактировать нельзя.

Для того, чтобы указать данный тип данных счетчик для Код_сотрудника необходимо выделить поле Код_сотрудника, перейти в свойства для данного поля и указать в **Спецификация идентификатора (Identity Specification)**:

- свойство **Идентификатор (Is Identity)** в Да (Yes);
- свойство **Начальное значение идентификатора** в 1;
- свойство **Шаг приращения идентификатора** в 1.

Имя столбца	Тип данных	Разрешить ...
Код_сотрудника	int	☐

Свойства столбца	
(Общие)	
Имя	Код_сотрудника
Значение по умолчанию или привязка	
Разрешить значения NULL	Нет
Тип данных	int
Конструктор таблиц	
RowGuid	Нет
Детерминированный	Да
Имеет подписчик, отличный от подписчика SQL Server	Нет
Индексируемый	Да
Набор столбцов	Нет
Не для репликации	Нет
Описание	
Опубликован слиянием	Нет
Опубликован через службы DTS	Нет
Параметры сортировки	<база данных по умолчанию>
Размер	4
Разряженный	Нет
Реплицировано	Нет
Сжатый тип данных	int
Спецификация вычисляемого столбца	
Спецификация идентификатора	
(Идентификатор)	Да
Начальное значение идентификатора	1
Шаг приращения идентификатора	1
Спецификация полнотекстового столбца	
	Нет

ИЛИ

	Column Name	Data Type	Allow Nulls
🔑	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐
	Пол	nvarchar(10)	☒
	Семейное_положение	nvarchar(15)	☒
	Дети	bit	☒

Column Properties	
Data Type	int
Default Value or Binding	
Table Designer	
Collation	<database default>
Computed Column Specification	
Condensed Data Type	int
Description	
Deterministic	Yes
DTS-published	No
Full-text Specification	No
Has Non-SQL Server Subscriber	No
Identity Specification	Yes
(Is Identity)	Yes
Identity Increment	1
Identity Seed	1
Indexable	Yes
Merge-published	No

5.2.2.2 Поля Фамилия, Имя, Отчество, Должность

Размер поля текстовый до 30 символов. Поля Фамилия, Имя, Должность - **обязательные** поля (запрет NULL значений), в поле Отчество NULL значения допустимы:

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐

5.2.2.3 Поле Пол

Размер поля текстовый до 10 символов. NULL значения допустимы:

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐
▶	Пол	nvarchar(10)	☒

Условие на значение: ‘мужской’ OR ‘женский’:

Для этого в **Проверочные ограничения (Check Constraints)** в строке **Выражение (Expression)** укажите условие:

Назовите ограничение в свойстве **Имя (Name)** в **СК_Сотрудники_Пол**.

5.2.2.4 Поле Семейное_положение

Размер поля текстовый до 15 символов. NULL значения допустимы:

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐
	Пол	nvarchar(10)	☒
▶	Семейное_положение	nvarchar(15)	☒

Условие на значение:

Семейное_положение IN ('Женат','Замужем','Холост','Не замужем')

Выражение проверочного ограничения

Выражение:

Семейное_положение IN ('Женат','Замужем','Холост','Не замужем')

OK Отмена

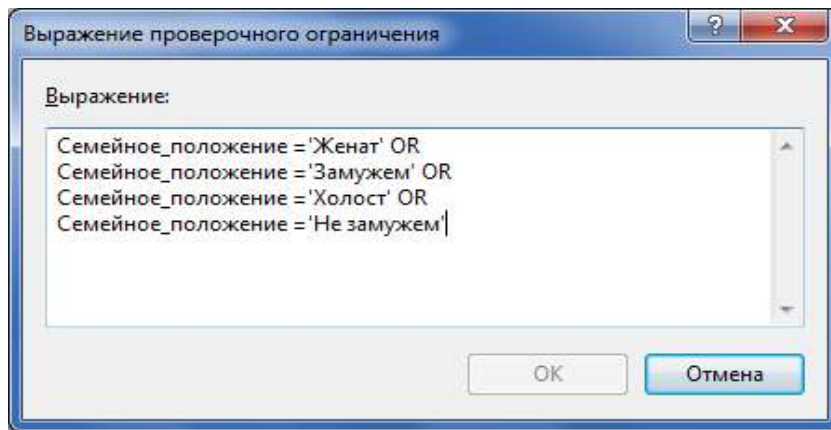
Check Constraint Expression

Expression:

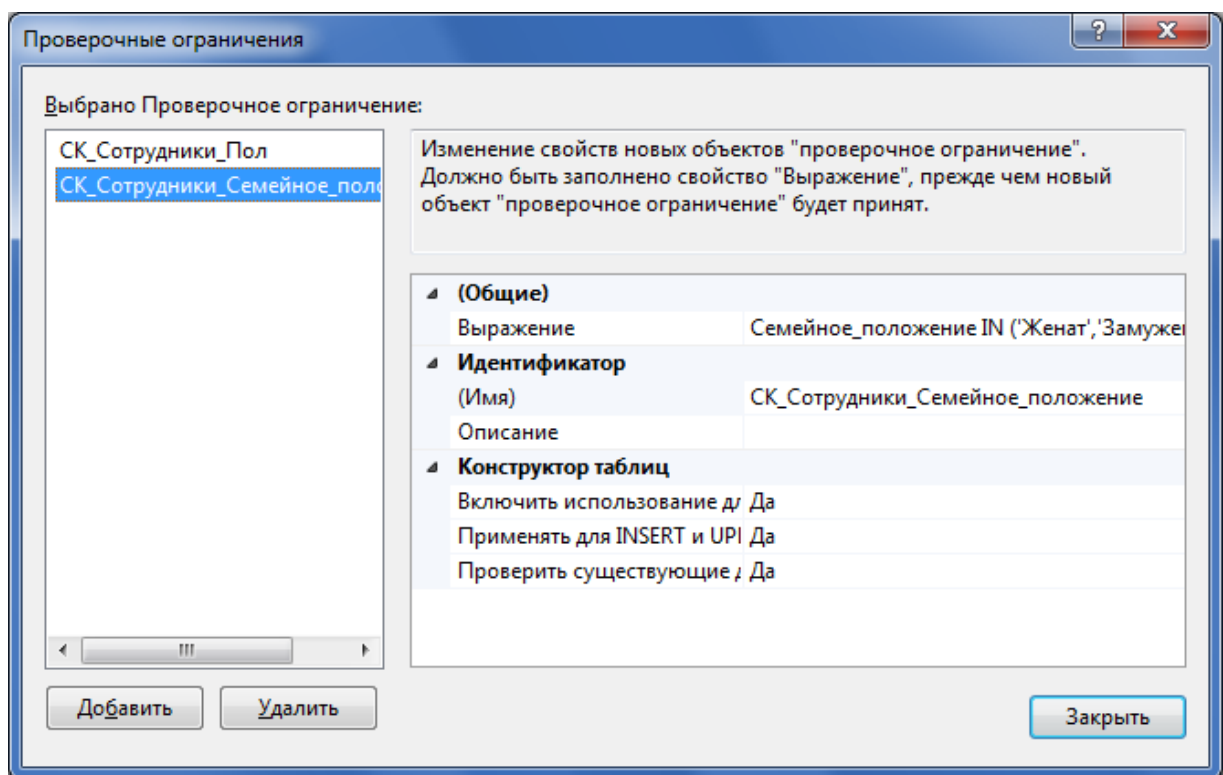
Семейное_положение IN ('Женат', 'Замужем', 'Холост', 'Не замужем')

OK Cancel

ИЛИ



Назовите ограничение в свойстве **Имя (Name)** в **СК_Сотрудники_Семейное_положение**.



5.2.2.5 Поле Дети

Выбран тип данных **bit** - **целочисленный тип данных**, который может принимать значения 1, 0 или NULL.

NULL значения для данного поля допустимы:

	Имя столбца	Тип данных	Разрешить значения NULL
?	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐
	Пол	nvarchar(10)	☒
	Семейное_положение	nvarchar(15)	☒
▶	Дети	bit	☒

5.2.2.6 Сохраните таблицу с именем **Сотрудники**.

5.2.2.7 Конструктор таблицы **Сотрудники** имеет вид:

	Имя столбца	Тип данных	Разрешить значения NULL
▶?	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐
	Пол	nvarchar(10)	☒
	Семейное_положение	nvarchar(15)	☒
	Дети	bit	☒

ИЛИ

	Column Name	Data Type	Allow Nulls
▶?	Код_сотрудника	int	☐
	Фамилия	nvarchar(30)	☐
	Имя	nvarchar(30)	☐
	Отчество	nvarchar(30)	☒
	Должность	nvarchar(30)	☐
	Пол	nvarchar(10)	☒
	Семейное_положение	nvarchar(15)	☒
	Дети	bit	☒

5.2.3 Таблица «Продажи»

Проведите создание таблицы **Продажи** аналогичным образом с необходимыми ограничениями.

Для таблицы **Продажи** задайте в конструкторе таблицы:

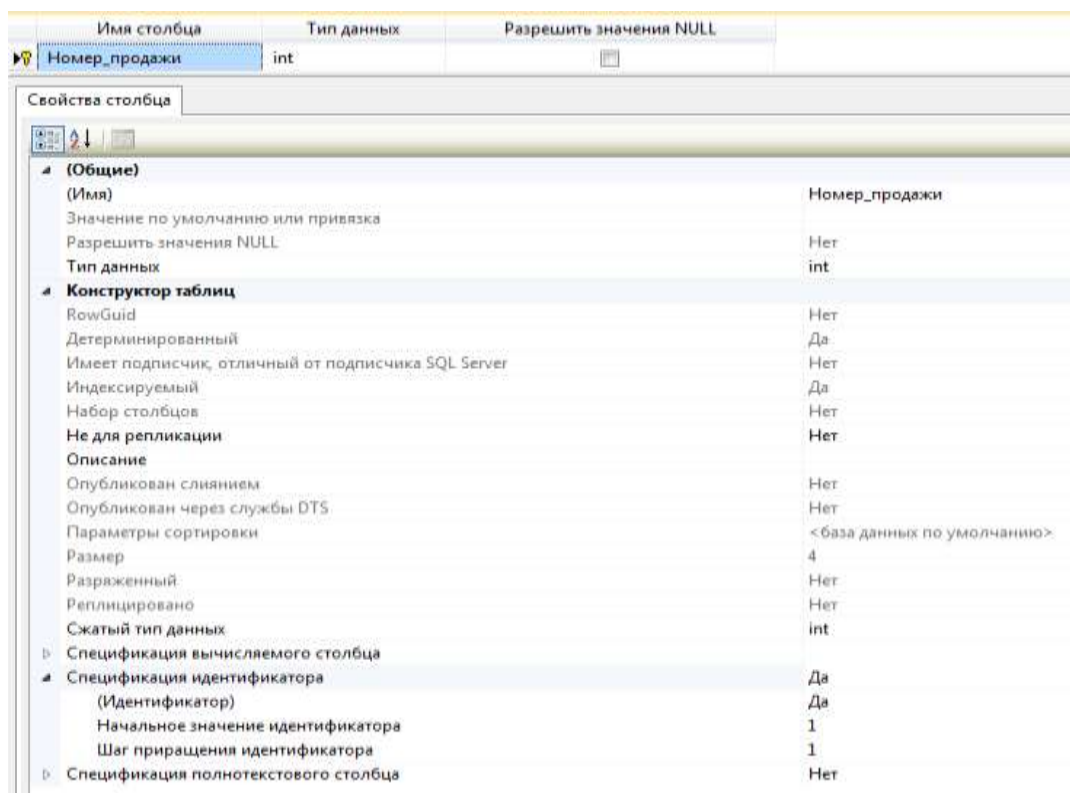
Имя столбца Column Name	Тип данных Data Type	Разрешить значения Nulls Allow Nulls
Номер_продажи	int	
Дата	smalldatetime	Флаг установлен
Код_сотрудника	int	
Номер_столика	nvarchar(10)	Флаг установлен

Теперь зададим необходимые ограничения для соответствующих полей.

5.2.3.1 Поле Номер_продажи

Данное поле **является первичным ключом**. Установите первичный ключ для поля Номер_продажи по аналогии с разделом 5.2.1.1.1.

Тип данных для данного поля решено установить в **счетчик (автоинкремент)**. Установите для Номер_продажи тип счетчик по аналогии с разделом 5.2.2.1.2.



5.2.3.2 Поле дата

Выбран тип данных **краткий формат даты**, **NULL значения допустимы**:

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Номер_продажи	int	☐
▶	Дата	smalldatetime	☒

5.2.3.3 Поле Код_сотрудника

5.2.3.3.1 Тип данных – целые числа, NULL значения не допустимы:

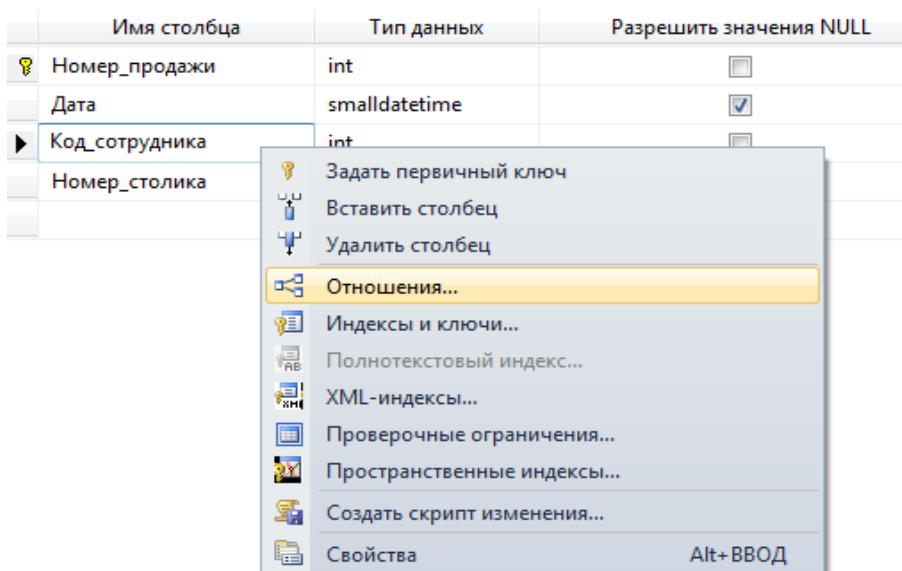
	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Номер_продажи	int	☐
	Дата	smalldatetime	☒
▶	Код_сотрудника	int	☐

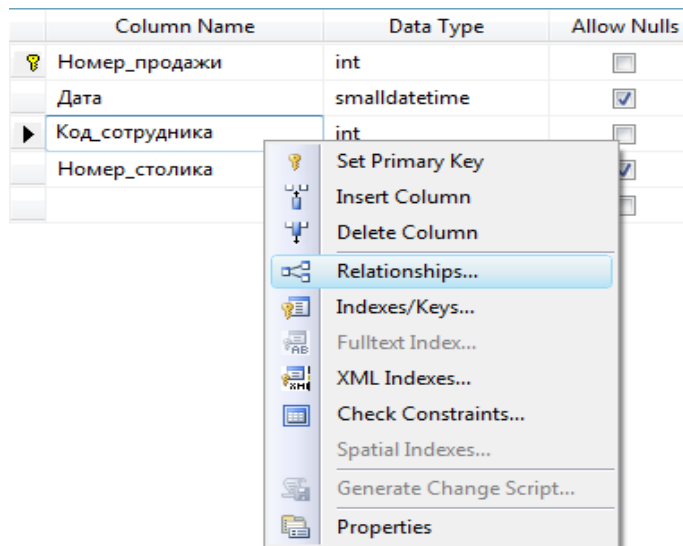
5.2.3.3.2 Внешний ключ

Данное поле является полем внешнего ключа. Родительской таблицей является таблица **Сотрудники**. Установить связь между таблицами **Сотрудники** и **Продажи** можно различными способами.

Создадим в данном случае связь через конструктор таблицы **Продажи**.

В меню конструктора таблицы **Продажи** нажмите кнопку связей/отношений (**Relationships**) или в конструкторе таблиц щелкните на кнопке **Отношения (Relationships)** панели инструментов:



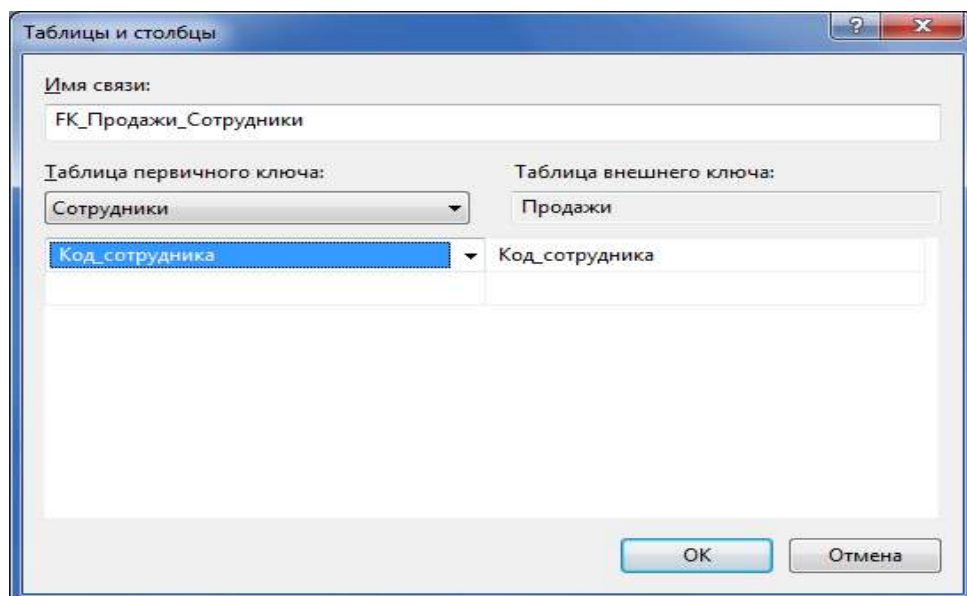


В диалоговом окне **Связи по внешнему ключу (Foreign Key Relationships)** нажмите кнопку **Добавить (Add)** для добавления связей.

Выделите **Спецификация таблиц и столбцов (Tables and Columns Specification)** и нажмите кнопку с многоточием (...) справа от свойства.

В появившемся диалоговом окне **Таблицы и столбцы (Tables and Columns)** укажите:

- **Таблица первичного ключа (Primary Key Table)** – родительскую таблицу **Сотрудники**;
- **Таблица внешнего ключа (Foreign Key Table)** – таблицу внешнего ключа **Продажи**;
- укажите, что связь устанавливается по **Код_сотрудника** в обеих таблицах:



ИЛИ

Relationship name:
FK_Продажи_Сотрудники

Primary key table:
Сотрудники
Код_сотрудника

Foreign key table:
Продажи
Код_сотрудника

OK Cancel

Нажмите кнопку ОК.

В результате в Спецификация таблиц и столбцов (Tables and Columns Specifications):

- Базовая таблица внешнего ключа (Foreign Key Base Table) – таблица Продажи;
- Базовая таблица первичного (уникального) ключа (Primary/Unique Key Base Table) – таблица Сотрудники;
- Внешние ключевые столбцы (Foreign Key Columns) – Код_сотрудника;
- Первичные (уникальные) столбцы (Primary/Unique Key Columns) – Код_сотрудника.

Связи по внешнему ключу

Выбрано Связи:
FK_Продажи_Сотрудники

Изменение свойств существующих объектов "связь".

▲ (Общие)
Проверить существующие данные при создании ил Да

▲ Спецификация таблиц и столбцов ...

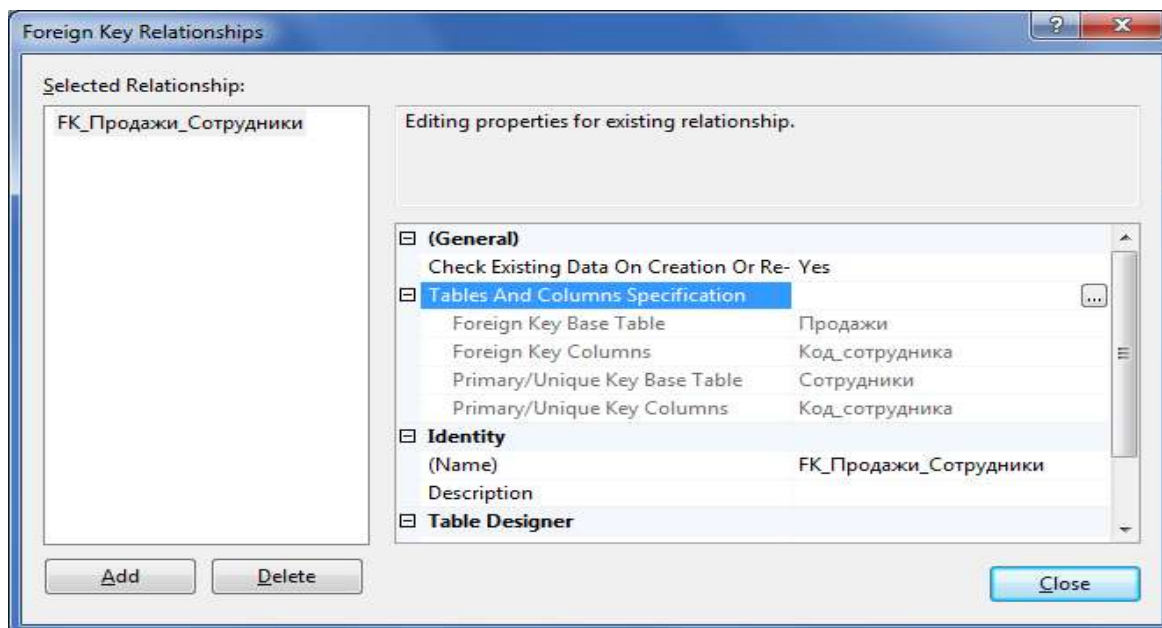
Базовая таблица внешнего ключа	Продажи
Базовая таблица первичного (уникального) клс	Сотрудники
Внешние ключевые столбцы	Код_сотрудника
Первичные (уникальные) столбцы	Код_сотрудника

▲ Идентификатор
(Имя) FK_Продажи_Со
Описание

▲ Конструктор таблиц

Добавить Удалить Закрыть

ИЛИ

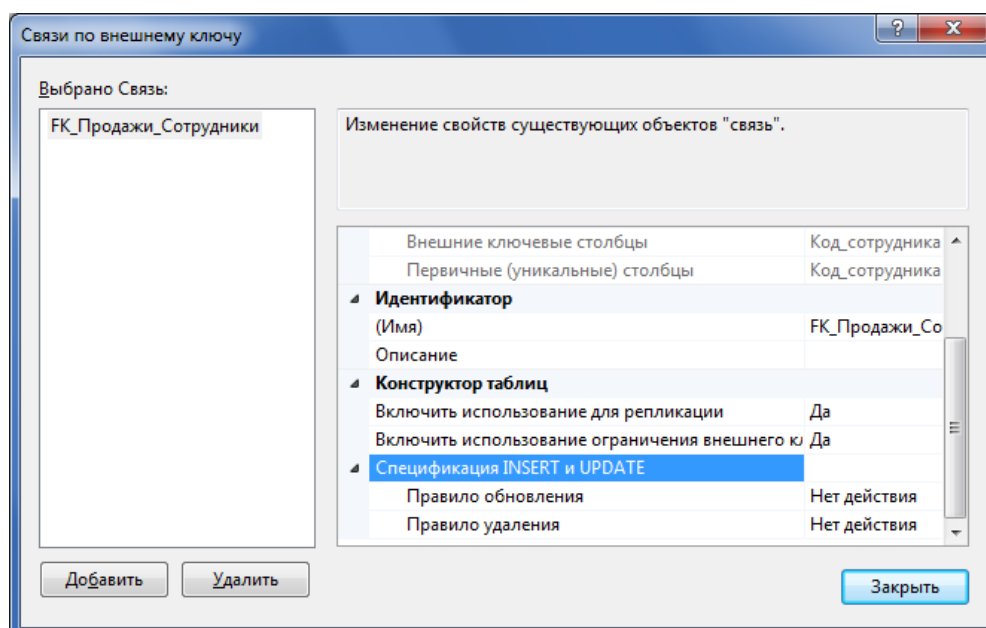


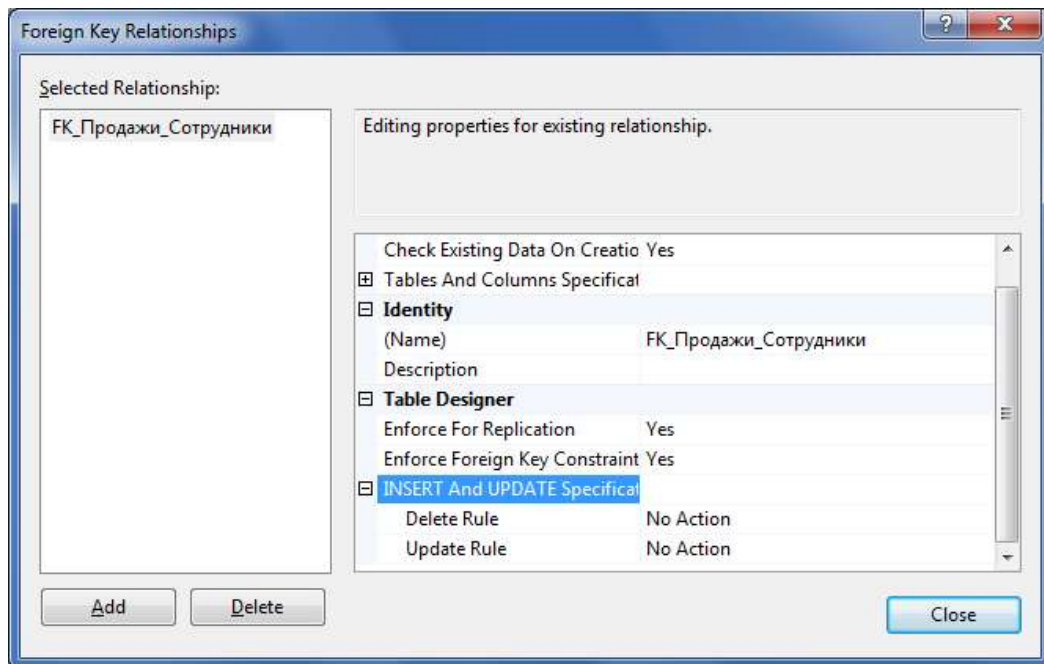
Для данной связи между таблицами **Сотрудники** и **Продажи** механизм **каскадного обновления связанных записей** не устанавливаем, так как Код_сотрудника в Сотрудники является типом данных счетчик, а его нельзя редактировать.

Для данной связи между таблицами **Сотрудники** и **Продажи** механизм **каскадного удаления связанных записей** не устанавливаем, чтобы при удалении данных о сотруднике, связанные с ним продажи автоматически не удалялись. Чтобы удалить сотрудника необходимо сначала удалить все его продажи, а затем сведения по самому сотруднику.

Таким образом, в **Спецификация INSERT и UPDATE (INSERT and UPDATE Specification)** в свойствах:

- **Правило обновления (Update Rule) - Нет действия (No Action);**
- **Правило удаления (Delete Rule) – Нет действия (No Action).**





5.2.3.4 Поле Номер_столика

Размер поля текстовый до 10 символов. NULL значения допустимы:

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Номер_продажи	int	☐
	Дата	smalldatetime	☒
	Код_сотрудника	int	☐
▶	Номер_столика	nvarchar(10)	☒

5.2.3.5 Сохраните таблицу с именем **Продажи**.

5.2.3.6 Конструктор таблицы **Продажи** имеет вид:

	Имя столбца	Тип данных	Разрешить значения NULL
▶🔑	Номер_продажи	int	☐
	Дата	smalldatetime	☒
	Код_сотрудника	int	☐
	Номер_столика	nvarchar(10)	☒

5.2.4 Таблица «Продано»

Проведите создание таблицы **Продано** аналогичным образом с необходимыми ограничениями.

Для таблицы **Продано** задайте в конструкторе таблицы:

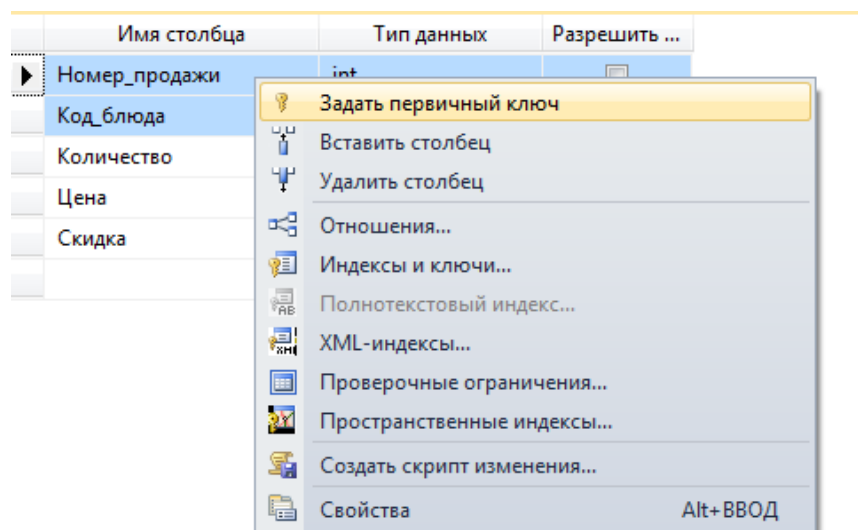
Имя столбца Column Name	Тип данных Data Type	Разрешить значения Nulls Allow Nulls
Номер_продажи	int	
Код_блюда	int	
Количество	int	
Цена	money	
Скидка	decimal(3,2)	Флаг установлен

Теперь зададим необходимые ограничения для соответствующих полей.

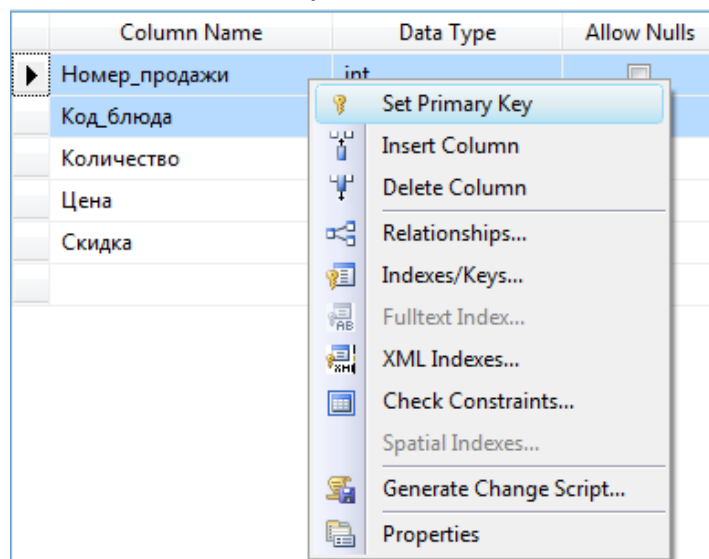
5.2.4.1 Поля Номер_продажи и Код_блюда

Оба этих поля входят в состав первичного ключа.

Выделите строки для Номер_продажи и Код_блюда, нажмите правую кнопку мыши и выберите **Задать первичный ключ (Set Primary Key)**:



ИЛИ



	Имя столбца	Тип данных	Разрешить значения NULL
	Номер_продажи	int	☐
	Код_блюда	int	☐

Кроме того, данные поля **являются внешними ключами**. Для данных полей можно установить связи по аналогии с внешним ключом Код_сотрудника в Продажи (раздел 5.2.3.3.2) или можно воспользоваться средствами диаграммы базы данных (сделаем так в данном случае, см. раздел 5.3).

5.2.4.2 Поле Количество

5.2.4.2.1 Тип данных – целые числа, NULL значения не допустимы:

	Имя столбца	Тип данных	Разрешить значения NULL
	Номер_продажи	int	☐
	Код_блюда	int	☐
	Количество	int	☐

5.2.4.2.2 Значение по умолчанию 1:

Аналогично как для поля Цена_меню (раздел 5.2.1.4.2), только в **Значения по умолчанию или привязка (Default Value or Binding)** укажите значение 1.

Имя столбца	Тип данных	Разрешить значения NULL
Номер_продажи	int	☐
Код_блюда	int	☐
Количество	int	☐

Свойства столбца

(Общие)

(Имя)

Количество

Значение по умолчанию или привязка

1

Разрешить значения NULL

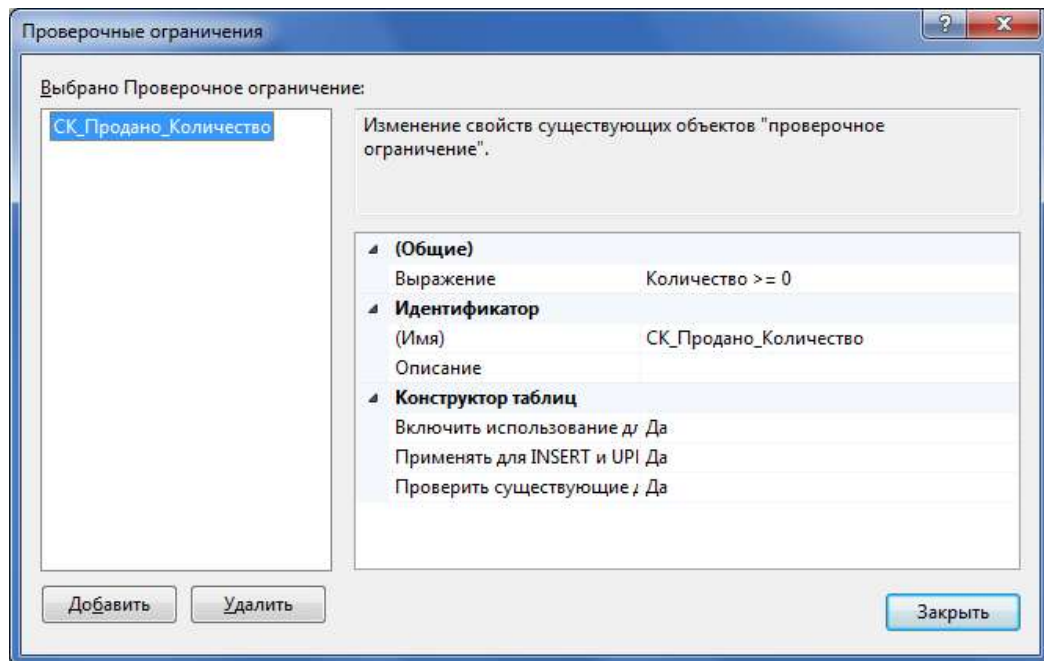
Нет

Тип данных

int

5.2.4.2.3 Условие на значение ≥ 0

Аналогично как для Код_блюда (раздел 5.2.1.1.2). Назовите ограничение **СК_Продано_Количество**.



5.2.4.3 Поле Цена

5.2.4.3.1 Тип данных – денежный, NULL значения не допустимы:

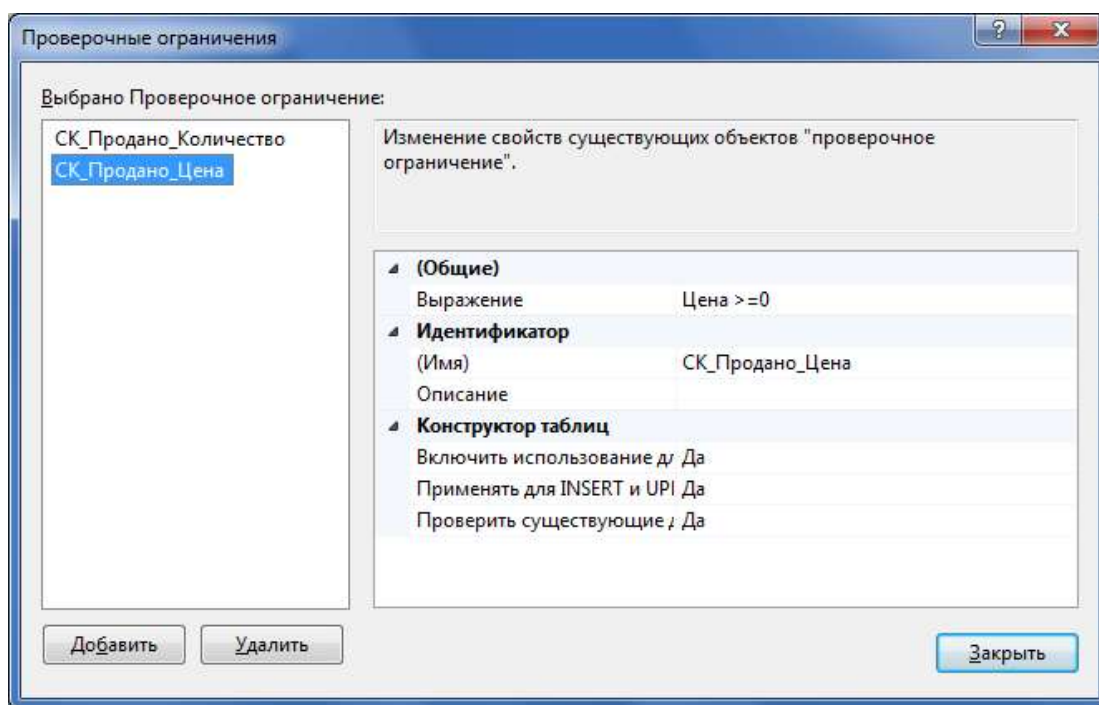
	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Номер_продажи	int	☐
🔑	Код_блюда	int	☐
	Количество	int	☐
▶	Цена	money	☐

5.2.4.3.2 Значение по умолчанию 0:

Аналогично как для поля Цена_меню (раздел 5.2.1.4.2).

5.2.4.3.3 Условие на значение ≥ 0

Аналогично как для Код_блюда (раздел 5.2.1.1.2).
Назовите ограничение **СК_Продано_Цена**.



5.2.4.4 Поле Скидка

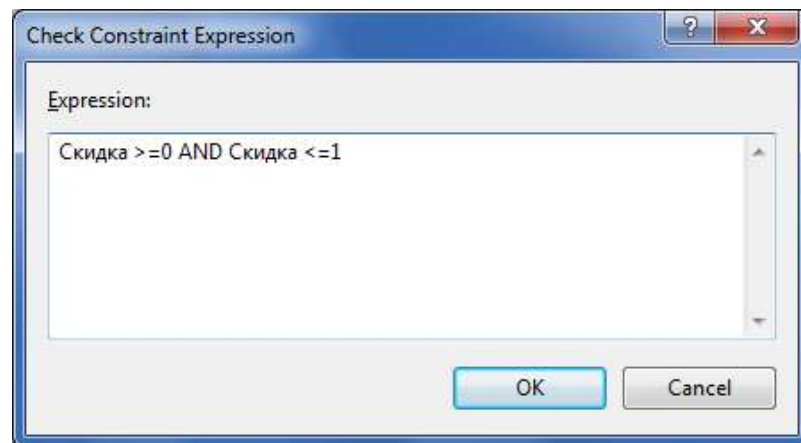
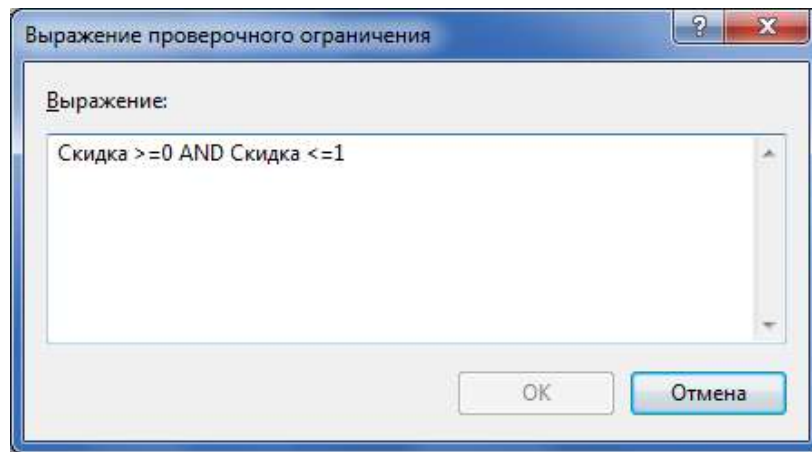
	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Номер_продажи	int	☐
🔑	Код_блюда	int	☐
	Количество	int	☐
	Цена	money	☐
▶	Скидка	decimal(3, 2)	☒

5.2.4.4.1 Значение по умолчанию 0

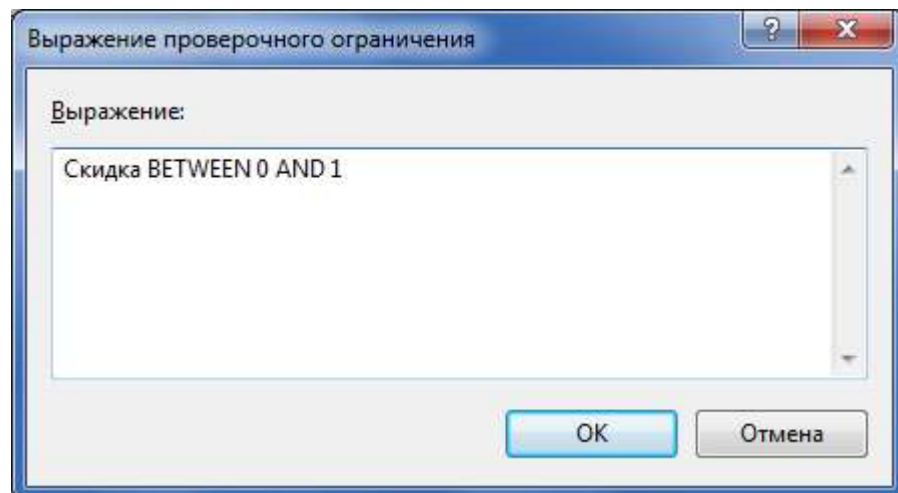
Аналогично как для поля Цена_меню (раздел 5.2.1.4.2).

5.2.4.4.2 Диапазон скидки от 0 до 1

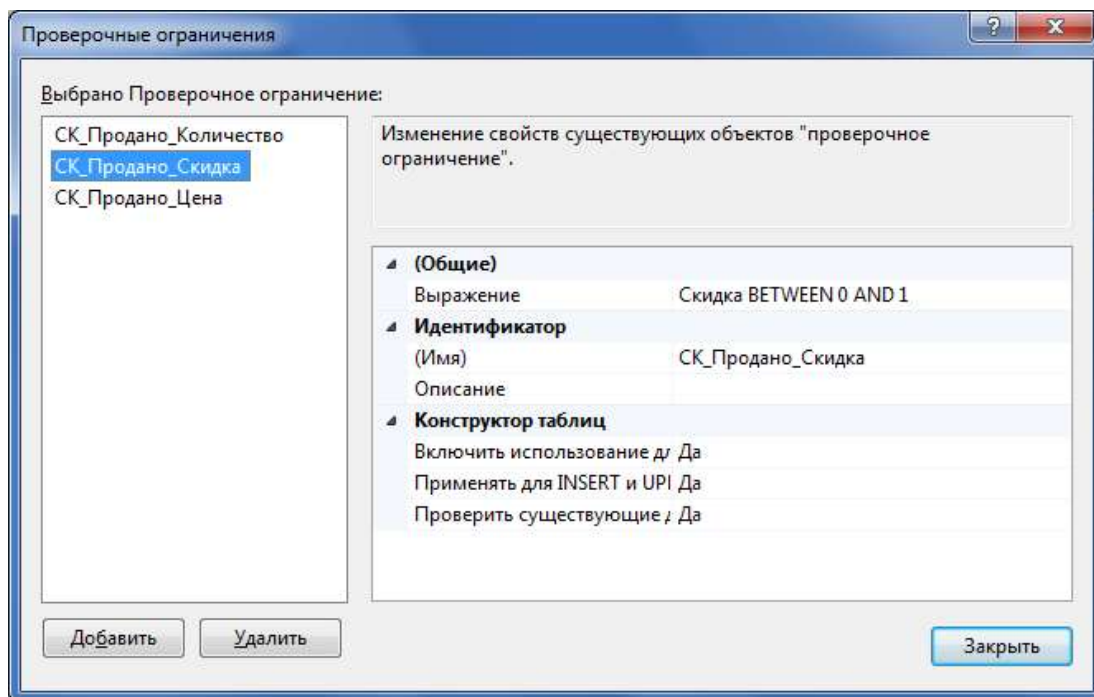
Аналогично как для Код_блюда (раздел 5.2.1.1.2), только в строке **Выражение (Expression)** укажите условие на значения **Скидка >=0 AND Скидка <=1**:



Или
Скидка BETWEEN 0 AND 1:



Назовите это ограничение **СК_Продано_Скидка**.



5.2.4.4.3 Тип данных для данного поля решено установить в **decimal(3,2)**, который позволит задать нам требуемую точность для скидок. В нашем случае диапазон скидки от 0 до 1, всего при записи скидки будут участвовать:

3 (точность) - максимальное количество десятичных разрядов числа (как слева, так и справа от десятичной запятой).

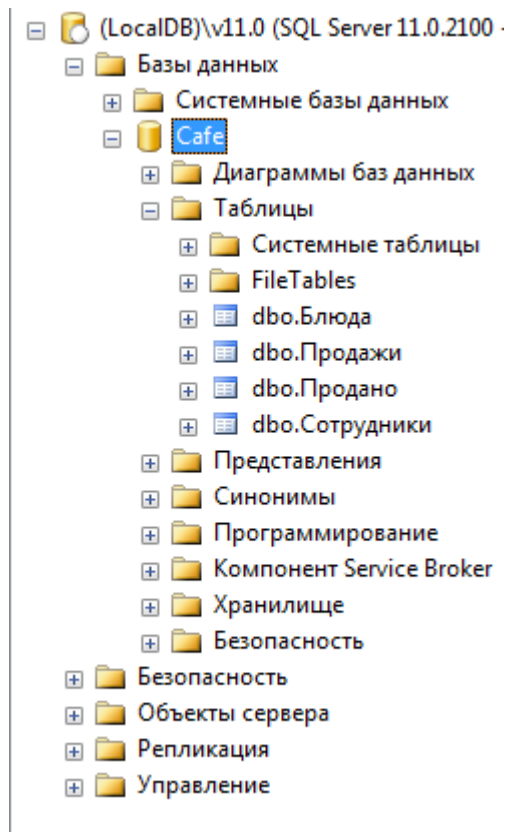
2 (масштаб) - максимальное количество десятичных разрядов числа справа от десятичной запятой.

5.2.4.5 Сохраните таблицу с именем **Продано**.

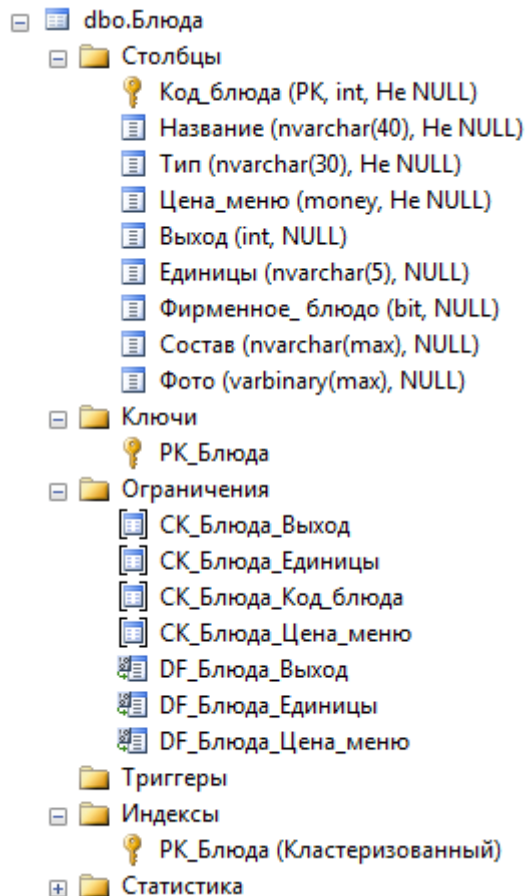
5.2.4.6 Конструктора таблицы **Продано** имеет вид:

	Имя столбца	Тип данных	Разрешить значения NULL
🔑	Номер_продажи	int	☐
🔑	Код_блюда	int	☐
	Количество	int	☐
	Цена	money	☐
	Скидка	decimal(3, 2)	☒

Таким образом, получаем таблицы:



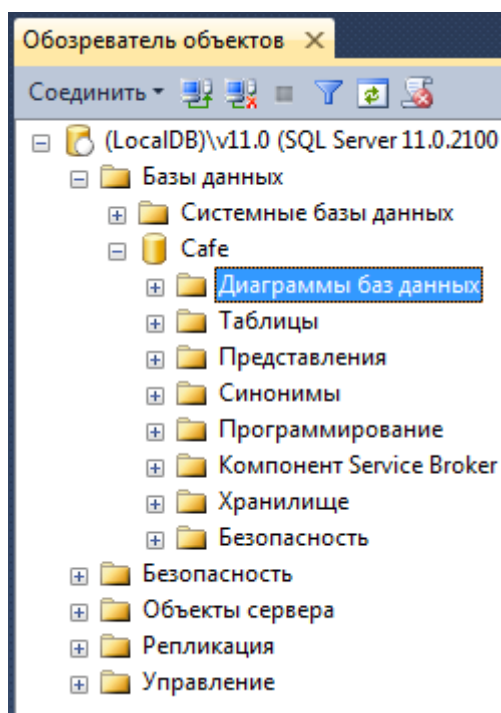
Например, для таблицы Блюда:



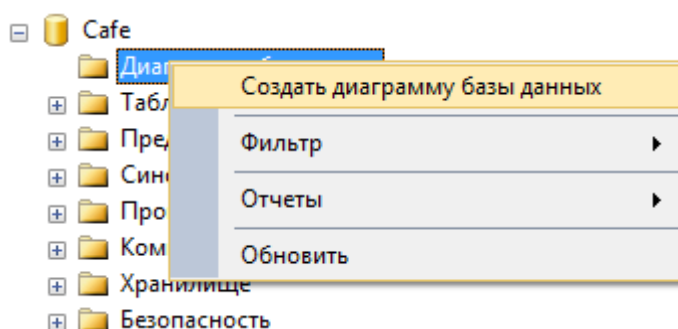
5.3 Создание диаграммы базы данных

Создадим диаграмму для нашей базы данных.

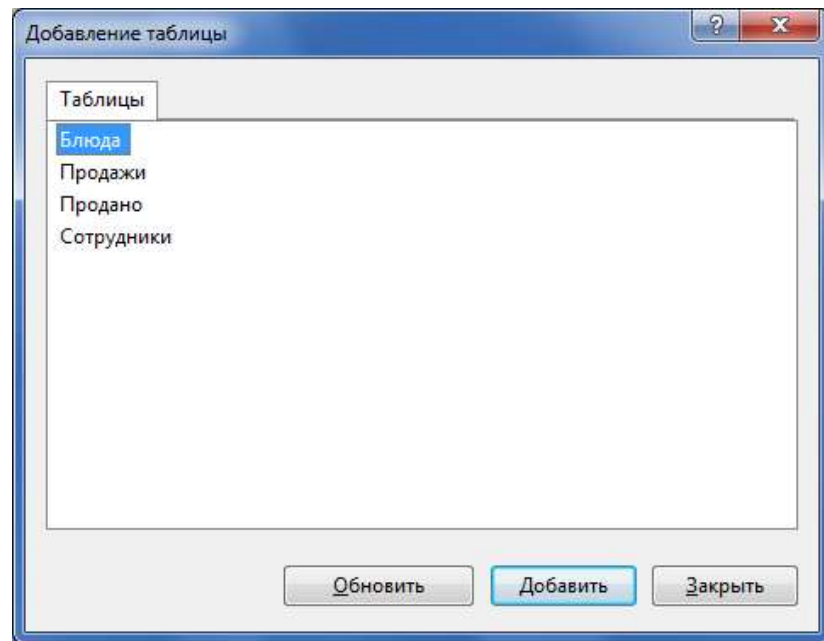
В окне **Обозреватель объектов** выделите узел **Диаграммы баз данных (Database Diagrams)**:



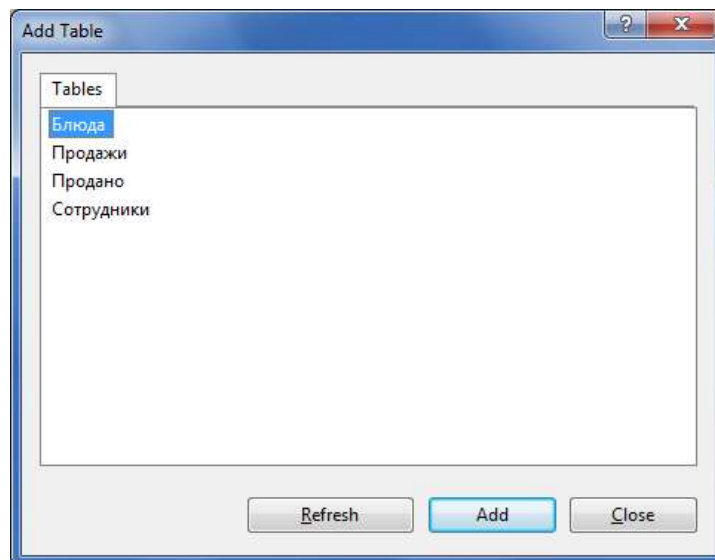
Нажмите правую кнопку на **Диаграммы баз данных (Database Diagrams)** и выберите пункт **Создать диаграмму базы данных**:



В появившемся диалоговом окне **Добавление таблицы (Add Table)** необходимо выбрать таблицы, добавляемые в диаграмму:

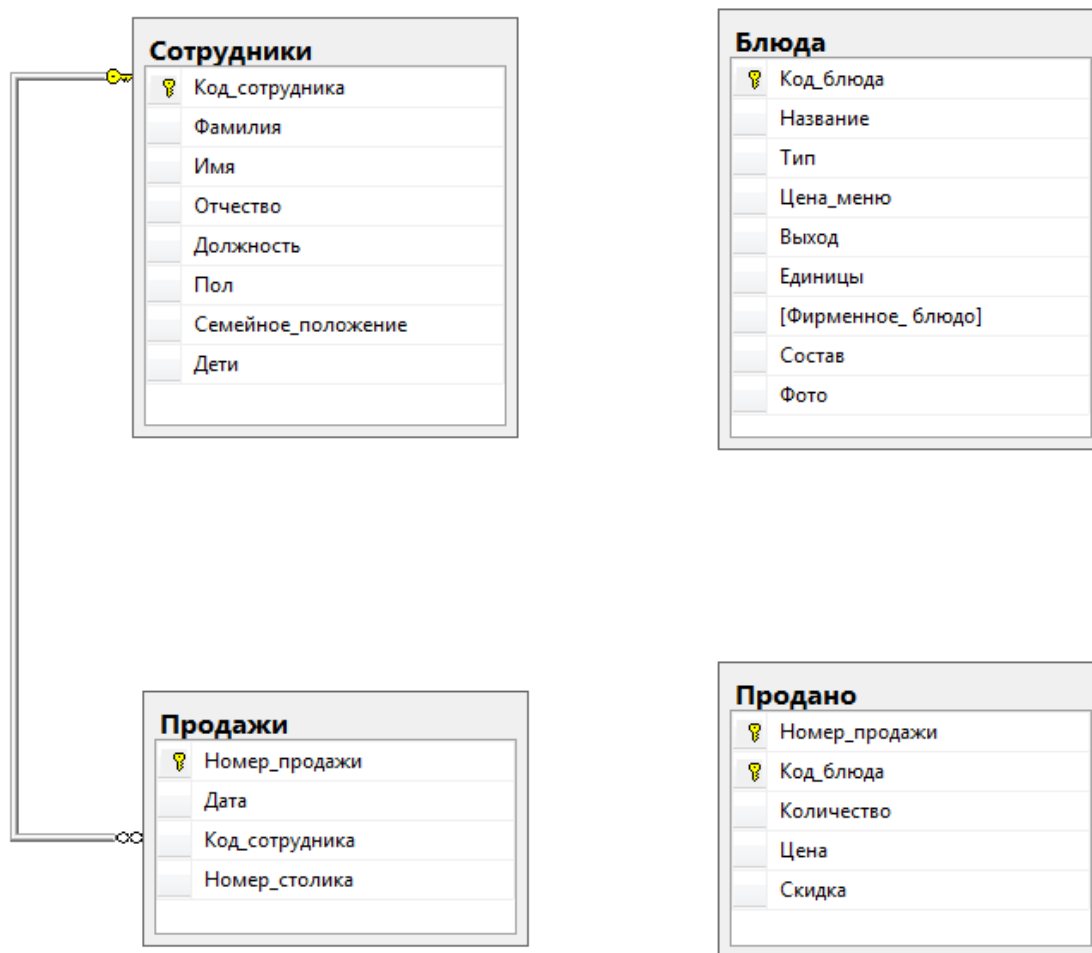


или



Выберете все таблицы БД.

Получаем в диаграмме:



Обратите внимание, что таблицы **Сотрудники** и **Продажи** уже связаны, так как данную связь мы установили в разделе 5.2.3.3.2.

Теперь установим связь между остальными таблицами БД согласно структуре на рисунке 4.1.

5.3.1 Установка связи между Продажи и Продано

Свяжите ключевое поле **Номер_продажи** простым перетаскиванием названия поля от родительской таблицы **Продажи** к дочерней таблице **Продано** по полю **Номер_продажи**:

В появившемся диалоговом окне проверьте правильность установки связи:

Таблицы и столбцы

Имя связи:
FK_Продано_Продажи

Таблица первичного ключа: Продажи
Номер_продажи

Таблица внешнего ключа: Продано
Номер_продажи

OK Отмена

Tables and Columns

Relationship name:
FK_Продано_Продажи

Primary key table: Продажи
Номер_продажи

Foreign key table: Продано
Номер_продажи

OK Cancel

Нажмите ОК, если связь устанавливается верно. В нашем случае по Номер_продажи.

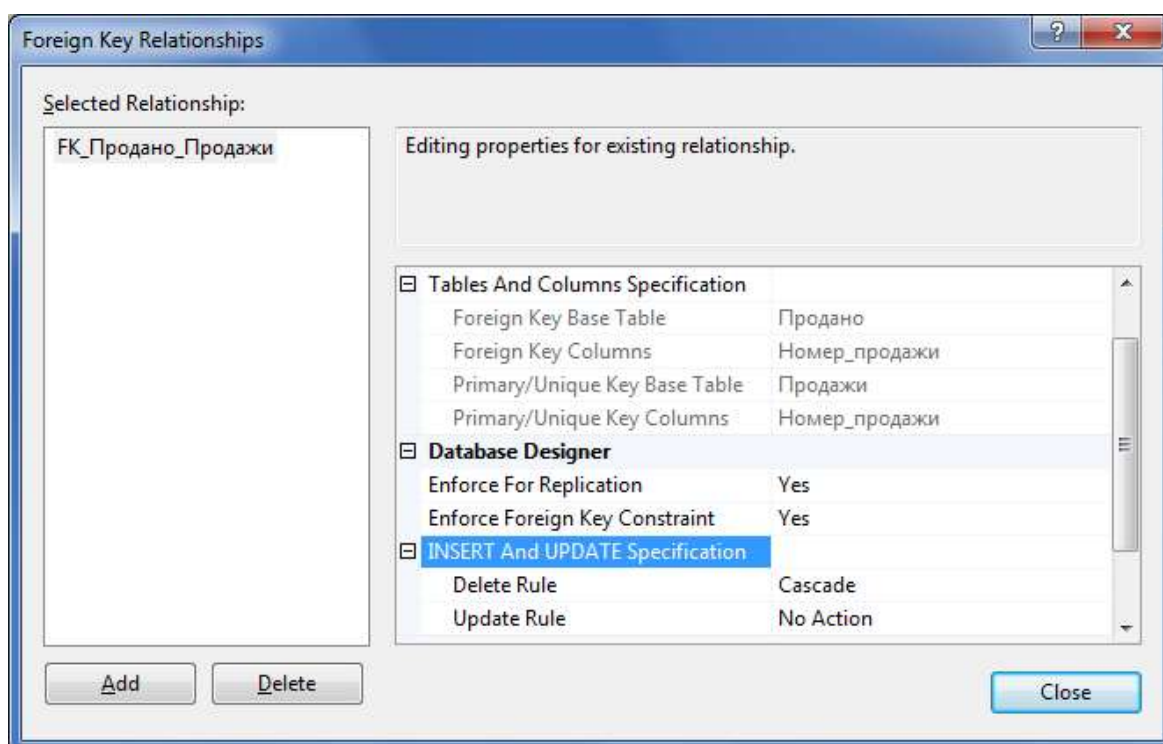
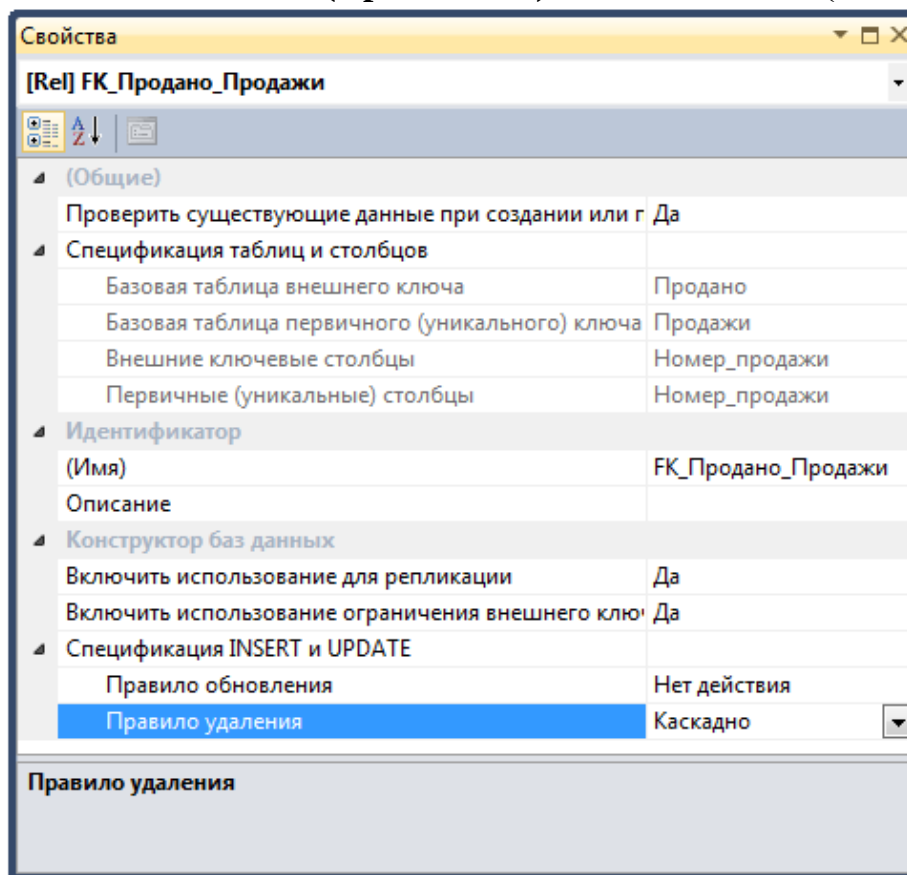
Для данной связи между таблицами **Продажи** и **Продано** механизм **каскадного обновления связанных записей** не устанавливаем, так как Номер_продажи в **Продажи** является типом данных счетчик, а его нельзя редактировать.

Устанавливаем для данной связи между таблицами **Продажи** и **Продано** механизм **каскадного удаления связанных записей**, чтобы при удалении данных о продаже, связанные с ней блюда в **Продано**, автоматически удалялись.

Таким образом, в Спецификация INSERT и UPDATE (INSERT and UPDATE Specification) в свойствах:

- Правило удаления (Delete Rule) – Каскадно (Cascade);

- Правило обновления (Update Rule) – Нет действия (No Action).



5.3.2 Установка связи между Блюда и Продано

По аналогии установите связь между **Блюда** и **Продано** по полю **Код_блюда**:

Таблицы и столбцы

Имя связи:
FK_Продано_Блюда

Таблица первичного ключа:
Блюда
Код_блюда

Таблица внешнего ключа:
Продано
Код_блюда

OK Отмена

Tables and Columns

Relationship name:
FK_Продано_Блюда

Primary key table:
Блюда
Код_блюда

Foreign key table:
Продано
Код_блюда

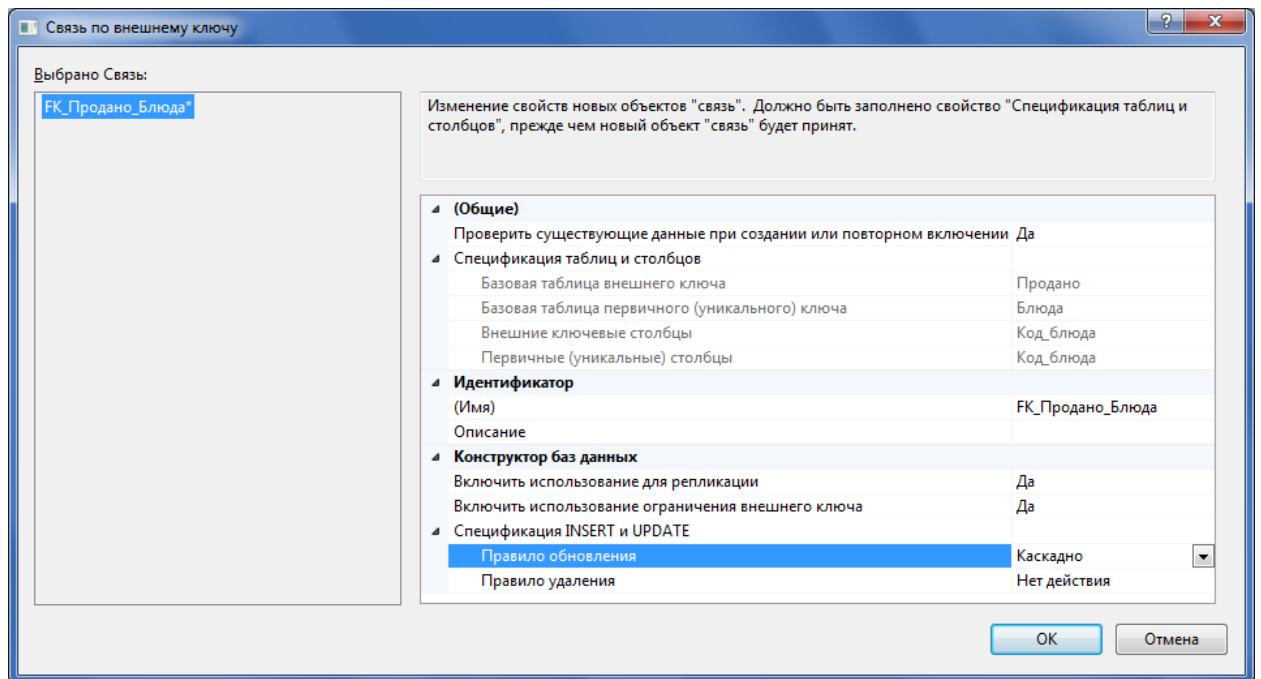
OK Cancel

Устанавливаем для данной связи между таблицами **Блюда** и **Продано** механизм **каскадного обновления связанных записей**, так как Код_блюда в **Блюда** может поменяться (например, введение новой формы меню, а блюдо неизменно). При этом необходимо, чтобы связанные с данным блюдом продажи автоматически среагировали на смену значения в Код_блюда.

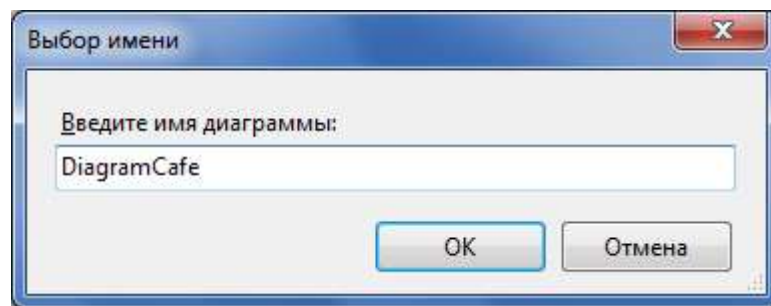
Механизм каскадного удаления связанных записей для данной связи не устанавливаем, чтобы при удалении данных о блюде, связанные с ним продажи в **Продано**, автоматически не удалялись.

Таким образом, в **Спецификация INSERT и UPDATE (INSERT and UPDATE Specification)** в свойствах:

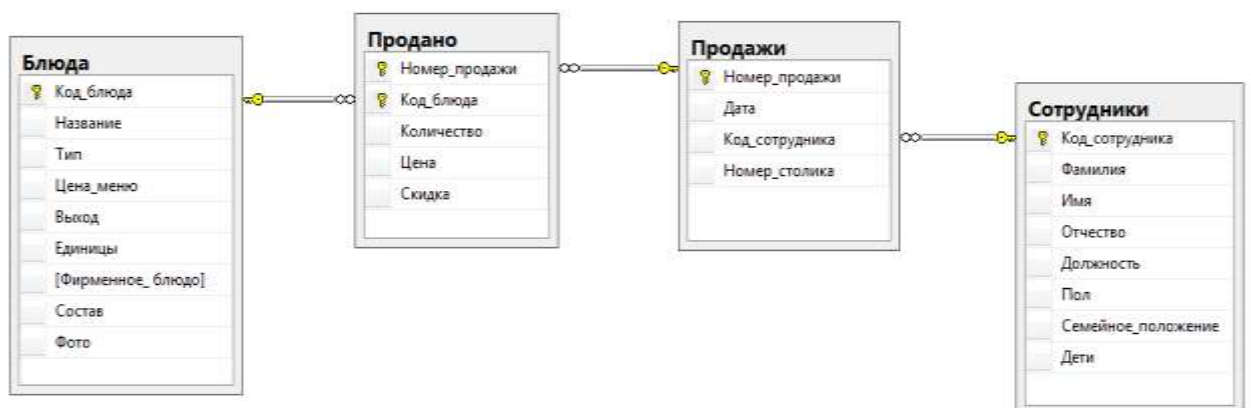
- Правило обновления (Update Rule) – Каскадно (Cascade);
- Правило удаления (Delete Rule) – Нет действия (No Action).



5.3.3 Сохраните полученную диаграмму с именем **DiagramCafe**.



5.3.4 В итоге получаем схему данных:



Что соответствует разработанной ранее логической модели, приведенной на рисунке 4.1.

5.4.2 Отследите обеспечение целостности

Например, в таблице **Блюда**.

При заполнении данных в данной таблицы были наложены ограничения целостности, нарушение которых будут приводить к соответствующим ошибкам.

Например:

- если повторить значения в первичном ключе **Код_блюда**;
- если оставить пустое значение в первичном ключе **Код_блюда**;
- в **Код_блюда**, **Цена_меню**, **Выход** были введены отрицательные числа;
- не заполнено **Название блюда**;
- не заполнен **Тип**, **Цена_меню**;
- в **Единицы** введено значение, отличное от допустимых "гр." или "мл." или "шт.";
- указан не соответствующий тип данных в **Код_блюда**, **Выход**;
- указан не соответствующий тип данных в **Цена_меню**;

- ссылочная целостность:

Данная таблица **Блюда** связана с таблицей **Продано**. Для данной связи было установлено обеспечение целостности данных, а также установлен механизм **каскадного обновления связанных записей**, так как Код_блюда в Блюдах может поменяться (например, введение новой формы меню, а блюдо неизменно). Механизм **каскадного удаления связанных записей** для данной связи не установлен, чтобы при удалении данных о блюде, связанные с ним продажи в **Продано**, автоматически не удалялись.

Нарушение данных условий приведет к следующим ошибкам:

- при удалении блюда в таблице Блюда, записи о котором есть в таблице Продано. Чтобы удалить блюдо в данном случае необходимо сначала удалить связанные записи в таблице Продано, а потом удалить блюдо из таблицы Блюда.

- если в таблице Продано указать не существующий в таблице Блюда Код_блюда.

Примечание:

посмотрите, что если для определенного блюда изменить его код, то он автоматически изменится в таблице Продано, так как был установлен механизм **каскадного обновления связанных записей**.

По аналогии опишите наложенные ограничения и для остальных таблиц в своей БД.

6 Оформление отчета к лабораторной работе

Оформить отчёт со следующим содержанием:

6.1 Титульный лист.

6.2 Введение (цель работы, постановка задачи).

6.3 Ход выполнения заданий и описание результатов.

Приведите скриншоты полученной схемы данных, конструкторов таблиц, режимов таблиц с заполненными данными. Опишите наложенные ограничения на Вашу БД. Покажите отработку ограничений целостности в Вашей БД.

6.4 Заключение (выводы).

Отчет должен быть оформлен согласно требованиям ОС ТУСУР.

7 Контрольные вопросы

- 7.1 Что такое база данных?
- 7.2 Что такое СУБД?
- 7.3 Что представляет собой реляционная таблица?
- 7.4 Какие ограничения можно наложить на данные в БД?
- 7.5 Что такое первичный ключ? Требования к первичному ключу.
- 7.6 Что такое внешний ключ?
- 7.7 Что характерно для ссылочной целостности?
- 7.8 Что подразумевает каскадное удаление и каскадное обновление?
- 7.9 Охарактеризуйте связь один к одному 1:1.
- 7.10 Охарактеризуйте связь один ко многим 1:M.
- 7.11 Охарактеризуйте связь многие ко многим M:M.
- 7.12 Какие режимы аутентификации поддерживает Microsoft SQL Server?
- 7.13 Как можно создать БД Microsoft SQL Server через SQL Server Management Studio?
- 7.14 Как можно создать таблицу Microsoft SQL Server?
- 7.15 Приведите примеры типов данных в Microsoft SQL Server.
- 7.16 Приведите примеры свойств полей в Microsoft SQL Server.
- 7.17 Приведите примеры операторов, использующихся для ограничений проверки.
- 7.18 Как можно установить связи между таблицами БД в Microsoft SQL Server?